מדריך מעשי למתחילים

ללמוד JavaScript ו-React דרך פרויקט אמיתי

מאפס ועד היכולת לקרוא, להבין ולשנות קוד Frontend — עם דוגמאות מתוך הפרויקט FlowBoard

חלק א׳ — JavaScript מהיסוד · חלק ב׳ — React, Router, TanStack Query ו-MUI

`JavaScript` `React 19` `React Router 7` `TanStack Query 5` `MUI 7` `Vite`

תוכן העניינים

מבוא

0 איך משתמשים במדריך הזה ומה תצטרכו 3

חלק א׳ — JavaScript

1 מפת הפרויקט: איפה נמצא הקוד ומה כל קובץ עושה 5

2 יסודות השפה: משתנים, טיפוסים ותנאים 8

3 פונקציות 11

4 אובייקטים, מערכים ושיטות המערך 14

5 מודולים: import ו-export 18

6 קוד אסינכרוני: Promise ו-async/await 20

7 הדפדפן: DOM, אירועים ואחסון מקומי 24

8 תרגילים וסיכום חלק א׳ 26

חלק ב׳ — React

9 קומפוננטות, JSX ו-props 27

10 state ואירועים: useState וטפסים 32

11 Effects והוקים נוספים: useEffect, useRef, useMemo 36

12 Context והוקים מותאמים אישית 40

13 ניווט עם React Router 43

14 נתונים מהשרת עם TanStack Query 47

15 ממשק עם MUI: קריאת sx, Grid ו-Stack 50

16 תבניות מתקדמות שתפגשו בפרויקט 53

17 סיור מודרך: קריאת דף שלם שורה אחר שורה 56

18 איך קוראים קוד זר: שיטה, דיבוג ומה ללמוד הלאה 60

נספחים

א דף עזר מהיר (Cheat Sheet) 63

ב מילון מונחים 66

ג פתרונות לתרגילים 67

איך משתמשים במדריך הזה

המטרה אינה לשנן תחביר. המטרה היא שתפתחו קובץ `.jsx` שמעולם לא ראיתם ותדעו להסביר מה הוא עושה.

למי המדריך מיועד

לכם, אם אתם יודעים לתכנת ברמה בסיסית בשפה כלשהי (משתנים, תנאים, לולאות, פונקציות — למשל מפייתון), אבל לא מכירים JavaScript ו-React. המדריך מניח **אפס** ידע בשניהם, ולא מניח ידע ב-HTML מעבר להבנה שדף אינטרנט בנוי מתגיות.

איך המדריך בנוי

המדריך מחולק לשני חלקים. **חלק א׳** מלמד את השפה JavaScript, ורק את מה שתפגשו בפועל בקוד של פרויקט מודרני. **חלק ב׳** מלמד את React ואת הספריות שנסמכות עליו בפרויקט FlowBoard: React Router לניווט, TanStack Query לנתוני שרת ו-MUI לרכיבי ממשק.

בכל פרק תמצאו ארבעה סוגי תיבות:

> **טיפ**
> קיצור דרך, עצה או הרגל טוב.

> **שימו לב**
> טעות נפוצה שמבלבלת מתחילים.

> **בפרויקט שלנו**
> קטע אמיתי מהקוד של FlowBoard, עם הנתיב של הקובץ. פתחו אותו בעורך ותראו אותו בהקשר.

> **נסו בעצמכם**
> תרגיל קצר. הפתרונות נמצאים בנספח ג׳.

מה צריך כדי לתרגל

- **דפדפן** (Chrome, Edge או Firefox). לחצו `F12` ועברו ללשונית **Console**: זה מקום מעולה להריץ שורות JavaScript ולראות מיד את התוצאה.
- **Node.js** (אם כבר מותקן אצלכם בגלל הפרויקט) — מריץ קבצי JavaScript מהטרמינל: `node file.js`.
- **עורך קוד** — VS Code מומלץ. הפרויקט עצמו נמצא בתיקייה `flowboard/frontend`.
- הפרויקט רץ מקומית: אחרי `npm install` ו-`npm run dev` הוא זמין בכתובת `http://127.0.0.1:5173`. כשאתם משנים קובץ והדף מתעדכן מיד — זו הדרך הטובה ביותר ללמוד.

שיטת הלימוד המומלצת

1. **קראו** את ההסבר.
2. **הקלידו** את הדוגמאות בקונסולה (לא העתקה — הקלדה. היא מכריחה אתכם לקרוא כל תו).
3. **שנו** משהו ותנבאו מה יקרה לפני שאתם מריצים.

4. **מצאו** את אותה תבנית בקוד של הפרויקט. זה השלב שהופך ידע תיאורטי ליכולת קריאה.

> **שימו לב: הכיוון בקוד**
> הטקסט העברי במדריך זורם מימין לשמאל, אבל קוד תמיד נכתב ונקרא משמאל לימין. לכן כל בלוקי הקוד מוצגים בכיוון שמאל-לימין, וקטעי קוד בתוך משפט עברי (כמו `const x = 1`) מסומנים בגופן ממוין.

מילה על התחביר בפרויקט

הפרויקט כתוב ב-JavaScript מודרני (ES2022+) עם סיומות `.js` ו-`.jsx`. סיומת `.jsx` פירושה שהקובץ מכיל **JSX** — כתיבת תגיות דמויות HTML בתוך JavaScript — שנלמד בחלק ב'. פרויקטים אחרים שתפגשו כתובים לפעמים ב-TypeScript (`.ts` / `.tsx`): זה אותו דבר עם הוספת טיפוסים, ומי שמבין את המדריך יבין גם אותם.

חלק א׳

JavaScript

השפה שכל דף אינטרנט מדבר. נלמד רק את מה שתפגשו בפועל: משתנים, פונקציות, אובייקטים, מערכים, מודולים וקוד אסינכרוני.

חלק א׳ · פרק 1

מפת הפרויקט

לפני שקוראים שורת קוד אחת, צריך לדעת איפה עומדים. הפרק הזה הוא מפה של תיקיית ה-Frontend.

מה זה Frontend

אפליקציית ווב מורכבת משני חלקים. ה-**Backend** (השרת) שומר נתונים במסד נתונים ועונה לבקשות. ה-**Frontend** הוא מה שרץ **בדפדפן של המשתמש**: הוא מציג מסכים, מגיב ללחיצות ושולח בקשות לשרת. בפרויקט FlowBoard ה-Frontend הוא אפליקציית **SPA** (Single Page Application): טוענים דף HTML אחד, ו-JavaScript מחליף את התוכן שלו בלי לטעון דפים חדשים.

בקשה: דפדפן (Frontend) ◀ HTTP + JSON ◀ שרת (Backend API) ◀ מסד נתונים

תשובה: מסד נתונים ◀ שרת (Backend API) ◀ JSON ◀ דפדפן

Frontend = הקוד שרץ בדפדפן, וזה מה שנלמד במדריך.

מבנה התיקייה `frontend/`

קובץ / תיקייה	תפקיד
package.json	רשימת החבילות (dependencies) שהפרויקט משתמש בהן, והפקודות (scripts) כמו `npm run dev`.
vite.config.js	הגדרות של Vite — הכלי שמריץ ומרכיב את האפליקציה.
index.html	דף ה-HTML היחיד. מכיל `<div id="root">` ושורה שטוענת את `src/main.jsx`.
src/main.jsx	**נקודת הכניסה.** כאן React "נתלה" על ה-div ומתחיל לצייר.
src/App.jsx	טבלת הנתיבים: איזה כתובת (URL) מציגה איזה מסך.
src/features/	המסכים עצמם, מחולקים לפי נושא: `projects`, `kanban`, `tasks`, `admin` ועוד.
src/components/common/	רכיבים קטנים לשימוש חוזר: `KpiCard`, `ProjectCard`, `PageHeader`.
src/layout/	המסגרת הקבועה של האתר: תפריט צד, סרגל עליון, פעמון התראות.
src/auth/	התחברות: הקשר המשתמש (`AuthContext`), הגנה על נתיבים ודפי כניסה.
src/api/	כל התקשורת עם השרת (`http.js`, `api.js`).
src/lib/	פונקציות עזר והוקים משותפים: `queries.js`, `permissions.js`, `format.js`.
src/theme/	צבעים, גופנים ועיצוב כללי של MUI.

> **טיפ: איך "ללכת לאיבוד" בקוד בצורה בריאה**
>
> כל קוד React מתחיל מ-`main.jsx` ← `App.jsx` ← מסך בתוך `features/` ← רכיבים בתוך `components/`. אם אתם מחפשים "איפה נכתב הטקסט שאני רואה במסך", חפשו אותו ב-VS Code (`Ctrl+Shift+F`) — כמעט תמיד תגיעו ישר לקובץ הנכון.

הקובץ package.json

קובץ ה-`package.json` הוא כרטיס הזהות של הפרויקט. שני חלקים בו חשובים לקורא:

frontend/package.json (מקוצר)

```
{
  "scripts": {
    "dev": "vite",              // npm run dev   -> מריץ שרת פיתוח
    "build": "vite build",      // npm run build -> יוצר גרסה לפרודקשן
    "lint": "eslint .",         // בודק איכות קוד
    "test": "vitest run"        // מריץ בדיקות
  },
  "dependencies": {
    "react": "^19.2.8",
    "react-dom": "^19.2.8",
    "react-router-dom": "^7.18.2",
    "@tanstack/react-query": "^5.102.4",
    "@mui/material": "^7.3.11",
    "axios": "^1.19.0",
    "recharts": "^3.10.1"
  }
}
```

- **scripts** — הפקודות שאפשר להריץ עם `npm run <שם>`.

- **dependencies** — ספריות חיצוניות. כשאתם רואים בקובץ `import ... from 'axios'` (בלי נקודה בתחילת הנתיב) — זו ספרייה מהרשימה הזו, שהתקנה ב-`node_modules`.

בפרויקט שלנו: נקודת הכניסה

קובץ `src/main.jsx` הוא הדוגמה הקצרה והחשובה ביותר. קראו אותו עכשיו — בפרק 12 נפרק אותו לגורמים.

src/main.jsx

```
createRoot(document.getElementById('root')).render(
  <StrictMode>
    <ErrorBoundary>
      <ThemeProvider theme={theme}>
        <QueryClientProvider client={queryClient}>
          <BrowserRouter>
            <AuthProvider>
              <App />
            </AuthProvider>
          </BrowserRouter>
        </QueryClientProvider>
      </ThemeProvider>
    </ErrorBoundary>
  </StrictMode>,
);
```

גם בלי לדעת כלום, אפשר לקרוא את זה כמו משפט: "צייר את `App` בתוך `AuthProvider`, בתוך `BrowserRouter` (ניווט), בתוך `QueryClientProvider` (נתוני שרת), בתוך `ThemeProvider` (עיצוב), בתוך `ErrorBoundary` (תפיסת שגיאות)". כל עטיפה נותנת ל-`App` ולכל מה שבתוכו "יכולת" נוספת.

יסודות השפה

משתנים, טיפוסים, מחרוזות, תנאים והשוואות. כל מה שקוד JavaScript בנוי ממנו.

משתנים: const ו-let

ב-JavaScript מודרני מגדירים משתנה עם אחת משתי מילים. `const` — הערך לא יוחלף בהמשך (זו הברירת מחדל, ותראו אותה ברוב הקוד). `let` — אפשר להציב בו ערך חדש.

```
const name = 'Dana';      // cannot be re-assigned
let count = 0;            // can be re-assigned
count = count + 1;        // ok
count += 1;               // same thing, shorter
count++;                  // add one

name = 'Yoni';            // TypeError: Assignment to constant variable
```

שימו לב: const לא אומר "קפוא"

`const` מונע **הצבה מחדש** של המשתנה, לא שינוי של אובייקט שהוא מחזיק. ל-`const list = []` אפשר להוסיף איברים (`list.push(1)`). בפרק 10 נראה למה React דווקא מעדיף לא לשנות אובייקטים.

הישן `var` כמעט לא בשימוש. אם אתם רואים אותו בקוד ותיק — התייחסו אליו כמו אל `let`.

טיפוסי הנתונים

טיפוס	דוגמאות	הערה
number	42, 3.14, -7, NaN	מספר אחד לכל הסוגים (שלם ועשרוני).
string	'hi', "hi", `hi`	טקסט. אפשר גרש בודד, כפול, או backtick.
boolean	true, false	אמת / שקר.
null	null	"בכוונה אין ערך".
undefined	undefined	"עוד לא הוגדר / לא נשלח". זה מה שמקבלים ממשתנה שלא אותחל או מאובייקט בלי המאפיין.
object	{ a: 1 }, [1,2], new Date()	אובייקטים, מערכים ופונקציות (הכול "אובייקט").

```
typeof 42            // 'number'
typeof 'x'           // 'string'
typeof true          // 'boolean'
typeof undefined     // 'undefined'
typeof null          // 'object'   (a famous historical quirk)
typeof [1, 2]        // 'object'
Array.isArray([1,2]) // true       (the right way to detect an array)
```

מחרוזות ותבניות (Template literals)

מחרוזת עם backticks (`` ` ``) יכולה להכיל ביטויים בתוך `${ }`, ולהתפרש על כמה שורות. זו הדרך המקובלת לחבר טקסט עם משתנים, ותראו אותה בכל מקום.

```
const user = 'Dana';
const tasks = 3;
const msg = `Hello ${user}, you have ${tasks} task${tasks === 1 ? '' : 's'}.`;
// 'Hello Dana, you have 3 tasks.'

'  FlowBoard '.trim()          // 'FlowBoard'
'a,b,c'.split(',')             // ['a', 'b', 'c']
'Hello'.toUpperCase()          // 'HELLO'
'Hello'.includes('ell')        // true
'Hello'.startsWith('He')       // true
'Hello'.slice(1, 3)            // 'el'
'Hello'.replace('l', 'L')      // 'HeLlo' (only the first)
'Hello'.length                 // 5
```

בפרויקט שלנו

ב-`KpiCard.jsx`: `` `kpiSpark-${label.replace(/[^a-zA-Z0-9]/g, '')}` `` — בונה מזהה ייחודי מהכותרת של הכרטיס. הביטוי `/[^a-zA-Z0-9]/g` הוא ביטוי רגולרי שמסיר כל תו שאינו אות או ספרה.

אופרטורים והשוואות

```
5 + 2   5 - 2   5 * 2   5 / 2   5 % 2   2 ** 3     // 7 3 10 2.5 1 8
'a' + 'b'                                           // 'ab'  (+ joins strings)
'5' + 2                                             // '52'  (!)  string wins
'5' - 2                                             // 3     (- converts to number)

5 === 5        // true    strict equality: same value AND same type
5 === '5'      // false
5 == '5'       // true    loose equality: converts types. AVOID.
5 !== 6        // true
5 > 3   5 <= 5
```

שימו לב: תמיד `===`

בפרויקט (ובכל קוד טוב) משתמשים ב-`===` ו-`!==`. ה-`==` הרגיל ממיר טיפוסים בדרכים מפתיעות (`0 == ''` הוא `true`). אם אתם רואים `==` — כנראה שזה קוד ישן.

Falsy ו-Truthy

בתנאי (`if`, `&&`, `?:`) JavaScript ממירה כל ערך ל-true או false. יש רק **שישה ערכי falsy**, כל השאר truthy:

```
// falsy:  false, 0, '' (empty string), null, undefined, NaN
// truthy: everything else - including '0', 'false', [] and {}

if ('hello') { /* runs */ }
if ([])      { /* runs! an empty array is truthy */ }
if (0)       { /* does not run */ }

!!'text'   // true   (double negation = "convert to boolean")
!!0        // false
```

תנאים

```
if (score >= 90) {
  grade = 'A';
} else if (score >= 80) {
  grade = 'B';
} else {
  grade = 'C';
}

// Ternary operator: condition ? valueIfTrue : valueIfFalse
const label = isManager ? 'Manager' : 'Employee';

// switch
switch (status) {
  case 'ACTIVE':   color = 'green'; break;
  case 'AT_RISK':  color = 'red';   break;
  default:         color = 'gray';
}
```

שלושת האופרטורים שתראו בכל שורה: `&&`, `||`, `??` ו-`?.`

אלה אופרטורים שמבלבלים בהתחלה ומופיעים בכל קובץ React, אז נקדיש להם זמן.

אופרטור	שם	משמעות
a && b	AND	אם `a` falsy — מחזיר `a`. אחרת מחזיר `b`. ("אם יש a, אז b")
a || b	OR	אם `a` truthy — מחזיר `a`. אחרת `b`. ("a, ואם אין אז b")
a ?? b	Nullish	מחזיר `b` **רק** אם `a` הוא `null` או `undefined` (לא אם הוא `0` או `''`).
a?.b	Optional chaining	אם `a` הוא null/undefined — מחזיר undefined במקום לזרוק שגיאה. אחרת `a.b`.

```
const user = null;
user.name          // TypeError: Cannot read properties of null
user?.name         // undefined  (no error)
user?.address?.city // undefined  (safe deep access)

const port = config.port ?? 3000;   // 0 is kept as 0; only null/undefined -> 3000
const title = input || 'Untitled';  // '' becomes 'Untitled' too

isLoggedIn && showMenu();           // runs showMenu only if isLoggedIn is truthy
```

בפרויקט שלנו

כמה שורות מהקוד האמיתי — נסו לתרגם כל אחת:

- `organization?.id` (ב-`queries.js`) — "ה-id של הארגון, אם כבר נטען ארגון".
- `(users ?? []).map(...)` (ב-`MyProjectsPage`) — "אם users עוד לא הגיעו, השתמש ברשימה ריקה".
- `import.meta.env.VITE_API_URL ?? 'http://localhost:3000/api'` (ב-`http.js`) — "כתובת השרת מהגדרות הסביבה, ואם אין — ברירת מחדל מקומית".

נסו בעצמכם 2.1

מה מדפיס כל אחד מהביטויים? נבאו לפני שאתם מריצים בקונסולה: `0 ?? 5` · `0 || 5` · `'' ?? 'x'` · `null?.a` · `[] && 'yes'` · `typeof NaN`.

פונקציות

ב-React כל רכיב הוא פונקציה, וכל אירוע מטופל בפונקציה. אין דרך לקרוא קוד React בלי להבין פונקציות.

שלוש דרכים לכתוב פונקציה

```
// 1. Function declaration
function add(a, b) {
  return a + b;
}

// 2. Function expression (a function stored in a variable)
const add2 = function (a, b) {
  return a + b;
};

// 3. Arrow function - the one you'll see most
const add3 = (a, b) => {
  return a + b;
};

// Arrow with an implicit return: no braces, no "return"
const add4 = (a, b) => a + b;

// One parameter: parentheses are optional
const double = (n) => n * 2;
const double2 = n => n * 2;

// No parameters
const now = () => Date.now();

add(2, 3)   // 5
```

איך לקרוא arrow function

קראו את החץ `=>` כמו "מעביר ל-": `(a, b) => a + b` אומר "a ו-b מועברים ל-a+b". אם אחרי החץ יש `{` — זה גוף עם פקודות ו-`return` מפורש. אם אין — הביטוי מוחזר אוטומטית.

שימו לב: החזרת אובייקט מ-arrow

כדי להחזיר אובייקט ישירות, עטפו אותו בסוגריים: `() => ({ id: 1 })`. בלי הסוגריים, ה-`{` ייחשב לתחילת גוף הפונקציה. תראו את זה הרבה בתוך `.map(...)`.

פרמטרים: ברירת מחדל, rest ו-spread

```
// Default parameter values
function greet(name = 'guest') {
  return `Hello ${name}`;
}
greet()          // 'Hello guest'
greet('Dana')    // 'Hello Dana'

// Rest: collect "the rest" of the arguments into an array
function sum(...numbers) {
  return numbers.reduce((total, n) => total + n, 0);
}
sum(1, 2, 3, 4)  // 10

// Spread: unpack an array into separate arguments
const nums = [5, 1, 9];
Math.max(...nums)   // 9  (same as Math.max(5, 1, 9))
```

בפרויקט שלנו

ב-`queries.js`: `export function useProjects(filters = {})` — אם קראו ל-`useProjects()` בלי ארגומנט, `filters` הוא אובייקט ריק. ברירות המחדל חוסכות בדיקות "האם הועבר ערך".

Destructuring בפרמטרים

כשמעבירים אובייקט לפונקציה, אפשר "לפרק" אותו כבר בחתימה. זו התבנית המרכזית ב-React, ולכן חשוב מאוד להכיר אותה:

```
// Without destructuring
function showUser(user) {
  return user.name + ' ' + user.age;
}

// With destructuring - take the properties you need
function showUser2({ name, age }) {
  return name + ' ' + age;
}

// With default values and renaming
function card({ label, accentColor = '#3454D1', value: amount }) {
  return `${label}: ${amount} (${accentColor})`;
}

showUser2({ name: 'Dana', age: 30 });
```

בפרויקט שלנו: החתימה של KpiCard

src/components/common/KpiCard.jsx

```
export function KpiCard({ label, value, icon, accentColor = '#3454D1', trend, helperText,
sparkline }) {
```

הפונקציה מקבלת **אובייקט אחד** ומפרקת אותו לשבעה משתנים, ולאחד מהם (`accentColor`) יש ערך ברירת מחדל. כך נראית כמעט כל חתימה של רכיב React. הקריאה המתאימה נראית:

`<KpiCard label="Active" value={5} />`.

פונקציות כערכים: callbacks

ב-JavaScript פונקציה היא ערך רגיל: אפשר לשמור אותה במשתנה, להעביר אותה כארגומנט ולהחזיר אותה. פונקציה שמועברת לפונקציה אחרת ש"תקרא לה מאוחר יותר" נקראת **callback**.

```
const numbers = [1, 2, 3];

// map receives a callback and calls it for every item
const doubled = numbers.map((n) => n * 2);      // [2, 4, 6]

// setTimeout calls the callback after a delay
setTimeout(() => console.log('one second passed'), 1000);

// You can pass a named function too
function isEven(n) { return n % 2 === 0; }
numbers.filter(isEven);                          // [2]

// Higher-order function: returns a function
function multiplier(factor) {
  return (n) => n * factor;
}
const triple = multiplier(3);
triple(5)   // 15
```

הדוגמה האחרונה מראה גם **closure**: הפונקציה הפנימית "זוכרת" את `factor` גם אחרי שהפונקציה החיצונית סיימה לרוץ. Closure הוא המנגנון שמאחורי כמעט כל handler ב-React (למשל `onClick={() => navigate('/x')}` "זוכר" את `navigate`).

Scope: איפה משתנה חי

```
const outer = 'visible everywhere below';

function demo() {
  const inner = 'visible only inside demo';
  if (true) {
    const block = 'visible only inside these braces';
    let alsoBlock = 1;
  }
  console.log(outer);   // ok
  console.log(inner);   // ok
  console.log(block);   // ReferenceError - the block ended
}
console.log(inner);     // ReferenceError
```

`const` ו-`let` חיים רק בתוך הבלוק `{ }` הקרוב. פונקציה פנימית רואה את משתני הפונקציה החיצונית שלה — אבל לא להפך.

הערה: את `this` תפגשו בעיקר בקוד ישן ובמחלקות. בקוד React מודרני כמעט לא משתמשים בו, ו-arrow function היא הבחירה הבטוחה ל-callbacks.

נסו בעצמכם 3.1

כתבו (א) פונקציית חץ `isAdult` שמקבלת גיל ומחזירה true/false; (ב) פונקציה `describe({ name, age = 18 })` שמחזירה מחרוזת תבנית; (ג) `makeCounter()` שמחזירה פונקציה שבכל קריאה מחזירה מספר גדול ב-1 מהקודם (רמז: closure).

אובייקטים, מערכים ושיטות המערך

הנתונים בכל אפליקציה הם אובייקטים ומערכים. שיטות המערך (`map`, `filter` ועוד) הן הכלי המרכזי לעבד אותם — ולהציג אותם.

אובייקטים

אובייקט הוא אוסף של זוגות **מפתח: ערך**.

```
const project = {
  id: 'p1',
  name: 'Atlas Customer Portal',
  status: 'ACTIVE',
  progress: 62,
  manager: { firstName: 'Daniel', lastName: 'Cho' },   // nested object
  members: ['daniel', 'priya'],                          // array inside
};

project.name              // 'Atlas Customer Portal'   (dot notation)
project['status']         // 'ACTIVE'                  (bracket notation)
project.manager.lastName  // 'Cho'

const key = 'progress';
project[key]              // 62   (dynamic key)

project.progress = 70;    // change a value
project.archived = false; // add a new property
delete project.archived;  // remove it

'name' in project         // true
Object.keys(project)      // ['id', 'name', 'status', ...]
Object.values(project)
Object.entries(project)   // [['id','p1'], ['name','Atlas...'], ...]
```

קיצורים שתראו כל הזמן

```
const id = 'p1';
const name = 'Atlas';

// Shorthand property: { id: id, name: name }  ->  { id, name }
const p = { id, name };

// Computed key (dynamic name in [])
const field = 'status';
const update = { [field]: 'AT_RISK' };     // { status: 'AT_RISK' }

// Method shorthand
const api = {
  get(id) { /* ... */ },                   // same as get: function (id) {...}
};
```

Destructuring של אובייקטים ומערכים

```
const { name, status } = project;              // name = ..., status = ...
const { name: projectName } = project;         // rename while destructuring
const { progress = 0, risk = 'LOW' } = project; // defaults (risk isn't there -> 'LOW')
const { manager: { firstName } } = project;    // nested

// Arrays are destructured by POSITION
const [first, second] = ['a', 'b', 'c'];        // first='a', second='b'
const [, , third] = ['a', 'b', 'c'];            // skip two -> third='c'

// The most important line in React (you'll meet it in chapter 10):
const [count, setCount] = useState(0);         // array destructuring
```

Spread על אובייקטים ומערכים: העתקה ועדכון "בלי לשנות"

האופרטור `...` מפזר את התוכן של אובייקט או מערך לתוך חדש. זה הדרך לייצר **עותק מעודכן** בלי לשנות את המקור — עיקרון שחיוני ב-React.

```
const a = { x: 1, y: 2 };
const b = { ...a, y: 99, z: 3 };     // { x: 1, y: 99, z: 3 }  - a is unchanged
// later keys win: y: 99 overrides y: 2

const list = [1, 2, 3];
const more = [...list, 4];           // [1, 2, 3, 4]
const pre  = [0, ...list];           // [0, 1, 2, 3]
const copy = [...list];              // shallow copy

// Merge objects (defaults + user overrides)
const settings = { theme: 'light', lang: 'en', ...userSettings };
```

בפרויקט שלנו

ב-`queries.js`: `realApi.fetchProjects({ organizationId: organization.id, ...filters })` — בונה אובייקט חדש שמכיל את מזהה הארגון ובנוסף את כל המסננים שהועברו.

מערכים

```
const nums = [10, 20, 30];

nums[0]            // 10        (indexes start at 0)
nums.length        // 3
nums[nums.length - 1]  // 30    (last item)
nums.at(-1)        // 30        (same, newer syntax)

nums.push(40);     // adds to the end  (MUTATES the array)
nums.pop();        // removes the last
nums.includes(20)  // true
nums.indexOf(20)   // 1
nums.join('-')     // '10-20-30'
nums.slice(1, 3)   // [20, 30]  (returns a copy, doesn't mutate)
[...nums].reverse()// reversed COPY  (reverse() itself mutates!)
```

שיטות המערך שחובה להכיר

שש השיטות האלה מופיעות כמעט בכל קובץ. הן **לא משנות** את המערך המקורי ומחזירות ערך חדש (חוץ מ-`sort`, שמשנה!).

שיטה	מה היא עושה	מחזירה
map(fn)	מחליפה כל איבר בתוצאת `fn` עליו	מערך חדש באותו אורך
filter(fn)	משאירה רק איברים ש-`fn` החזירה עבורם truthy	מערך קצר יותר
find(fn)	מוצאת את האיבר הראשון שמתאים	איבר או `undefined`
some(fn) / every(fn)	האם **לפחות אחד** / **כולם** מתאימים	true / false
reduce(fn, init)	מצמצמת את המערך לערך יחיד (סכום, אובייקט...)	ערך כלשהו
forEach(fn)	מריצה `fn` על כל איבר (פעולה, בלי ערך חוזר)	undefined

```
const tasks = [
  { id: 1, title: 'Design',  done: true,  hours: 3 },
  { id: 2, title: 'Build',   done: false, hours: 8 },
  { id: 3, title: 'Test',    done: false, hours: 5 },
];

tasks.map((t) => t.title)                // ['Design', 'Build', 'Test']
tasks.map((t) => ({ ...t, hours: t.hours * 2 }))   // new objects, hours doubled
tasks.filter((t) => !t.done)             // [Build, Test]
tasks.find((t) => t.id === 2)            // { id: 2, title: 'Build', ... }
tasks.some((t) => t.done)                // true
tasks.every((t) => t.done)               // false
tasks.reduce((sum, t) => sum + t.hours, 0)// 16

// Chaining: open tasks -> their titles -> uppercased
tasks.filter((t) => !t.done).map((t) => t.title.toUpperCase());  // ['BUILD','TEST']

tasks.forEach((t) => console.log(t.title));
```

בפרויקט שלנו: filter + length

src/features/projects/MyProjectsPage.jsx

```
const summary = useMemo(() => {
  const list = projects ?? [];
  return {
    total: list.length,
    active: list.filter((p) => p.status === 'ACTIVE').length,
    atRisk: list.filter((p) => p.status === 'AT_RISK').length,
    completed: list.filter((p) => p.status === 'COMPLETED').length,
  };
}, [projects]);
```

מספרים על כל סטטוס: מסננים את הרשימה לפי סטטוס ולוקחים את האורך. (את `useMemo` נלמד בפרק 11 — כרגע התעלמו ממנו וקראו רק את גוף הפונקציה.)

Map/Set-ו sort

```
const nums = [10, 9, 100, 1];
nums.sort();                       // [1, 10, 100, 9]  -> sorts as TEXT by default!
[...nums].sort((a, b) => a - b);  // [1, 9, 10, 100]  numeric ascending (copy first)
[...nums].sort((a, b) => b - a);  // descending

people.sort((a, b) => a.name.localeCompare(b.name));  // sort by text

// Map: key -> value, any key type, fast lookup
const byId = new Map(users.map((u) => [u.id, u]));
byId.get('u1');                    // the user with id 'u1'
byId.has('u9');                    // false

// Set: unique values
const unique = new Set([1, 2, 2, 3]);   // {1, 2, 3}
[...unique];                            // [1, 2, 3]
```

בפרויקט שלנו

הופך — `const userMap = useMemo(() => new Map((users ?? []).map((u) => [u.id, u])), [users])` רשימת משתמשים למילון לפי id, כדי שאפשר יהיה לשלוף משתמש לפי מזהה ב-`userMap.get(project.managerId)`.

JSON

JSON הוא פורמט טקסט להעברת נתונים, בנוי כמו אובייקטי JavaScript (אבל מפתחות תמיד במרכאות כפולות). השרת שולח JSON והדפדפן ממיר אותו לאובייקטים.

```
const text = '{"name":"Dana","tags":["a","b"]}';
const obj = JSON.parse(text);          // text -> object
obj.tags[1]                            // 'b'

const back = JSON.stringify(obj);      // object -> text
JSON.stringify(obj, null, 2);          // pretty-printed with 2-space indent

localStorage.setItem('x', JSON.stringify({ a: 1 }));   // only strings can be stored
```

נסו בעצמכם 4.1

נתון המערך `orders = [{id:1, total:50, paid:true}, {id:2, total:120, paid:false}, {id:3, total:75, paid:false}]`

כתבו ביטוי אחד לכל סעיף: (א) סכום ההזמנות שלא שולמו; (ב) מערך של ה-id בלבד; (ג) האם יש הזמנה מעל 100; (ד) אובייקט `{ 1: 50, 2: 120, 3: 75 }` (רמז: `reduce` או `Object.fromEntries`).

מודולים: import ו-export

כל קובץ הוא מודול עצמאי. שורות ה-`import` בראש הקובץ הן "רשימת הקניות" שלו — וקריאה נכונה שלהן חוסכת חצי מהבלבול.

למה מודולים

במקום קובץ ענק אחד, מחלקים לקבצים קטנים. כל קובץ **מייצא** (`export`) את מה שהוא רוצה לחשוף, וקבצים אחרים **מייבאים** (`import`) אותו. מה שלא יוצא — נשאר פרטי לקובץ.

שני סוגי export

lib/permissions.js - named exports (as many as you like)

```
export const ROLE_LEVEL = { EMPLOYEE: 1, PROJECT_MANAGER: 2 };

export function isManager(role) {
  return ROLE_LEVEL[role] >= ROLE_LEVEL.PROJECT_MANAGER;
}
```

App.jsx - default export (at most one per file)

```
export default function App() {
  return <Routes>...</Routes>;
}
```

importing each kind

```
// Named export -> curly braces, the exact same name
import { ROLE_LEVEL, isManager } from './lib/permissions';

// Default export -> no braces, you choose the name
import App from './App.jsx';

// Rename a named import
import { isManager as checkManager } from './lib/permissions';

// Import everything as one object
import * as realApi from '../api/api';
realApi.fetchProjects();

// Side-effect import (just runs the file, e.g. loads CSS)
import './index.css';
```

איך לקרוא נתיב import

כך נראה	מה זה
`'./KpiCard'`	קובץ באותה תיקייה (הסיומת `.jsx` אפשר להשמיט).
`'../lib/queries'`	עולים תיקייה אחת (`..`) ואז נכנסים ל-`lib`.
`'../../auth/AuthContext'`	עולים שתי תיקיות.
`'react'`	בלי נקודה בהתחלה = **חבילה** מ-`node_modules` (ספרייה חיצונית).
`'@mui/material/Box'`	חבילה עם "scope" (`@mui`) ותת-נתיב של רכיב ספציפי.

בפרויקט שלנו: לקרוא את ה-imports של MyProjectsPage

src/features/projects/MyProjectsPage.jsx (imports)

```
import { useMemo, useState } from 'react';                      // React hooks (package)
import Box from '@mui/material/Box';                            // MUI component (default)
import { useNavigate } from 'react-router-dom';                 // router hook (named)
import { PageHeader } from '../../components/common/PageHeader'; // our own component
import { useAuth } from '../../auth/AuthContext';               // our own context hook
import { useProjects } from '../../lib/queries';                // our own data hook
import { isManager } from '../../lib/permissions';              // our own helper
```

רק מקריאת ה-imports אפשר לנחש: הדף משתמש ב-state, ניווט, נתוני משתמש מחובר, נתוני פרויקטים והרשאות. **תמיד קראו imports ראשונים** — זו תקציר של התלויות.

Dynamic import ו-lazy loading

לפעמים לא רוצים לטעון קובץ עד שצריך אותו. `import()` כפונקציה מחזיר Promise (פרק 6) ומאפשר "טעינה עצלה" — כך שהמשתמש מוריד רק את הקוד של המסך שהוא פותח.

בפרויקט שלנו

src/App.jsx

```
const lazyNamed = (loader, name) =>
  lazy(() => loader().then((module) => ({ default: module[name] })));

const MyProjectsPage = lazyNamed(
  () => import('./features/projects/MyProjectsPage'),
  'MyProjectsPage',
);
```

קראו: "`MyProjectsPage` נטען רק כשמישהו מבקש אותו. `lazyNamed` היא עזר קטן שמתאים מודול עם named export לפורמט ש-`lazy` דורש (default)". בפרק 13 נראה את זה בפעולה.

שימו לב: מעגלי תלות

אם קובץ A מייבא את B ו-B מייבא את A, תקבלו באגים מוזרים (ערכים שהם `undefined`). הפתרון: להוציא את הקוד המשותף לקובץ שלישי.

קוד אסינכרוני

בקשה לשרת לוקחת זמן. JavaScript לא מחכה — היא ממשיכה לרוץ ומטפלת בתוצאה כשהיא מגיעה. זה הנושא שמבלבל הכי הרבה, ואי אפשר בלעדיו.

הבעיה: JavaScript רצה על "חוט" אחד

דפדפן מריץ את ה-JavaScript של הדף בחוט אחד. אם נחכה שתשובה מהשרת תגיע (שנייה?), כל הדף יקפא. לכן פעולות איטיות (רשת, טיימרים, קבצים) הן **אסינכרוניות**: מתחילים אותן, ממשיכים הלאה, ומקבלים הודעה כשהן מסתיימות.

```
console.log('1 - start');
setTimeout(() => console.log('2 - after one second'), 1000);
console.log('3 - end');
```

```
▸ פלט
1 - start
3 - end
2 - after one second
```

שימו לב לסדר: השורה השלישית מודפסת **לפני** השנייה, כי `setTimeout` רק מתזמן את הפונקציה לעתיד.

Promise: הבטחה לתוצאה

Promise הוא אובייקט שמייצג תוצאה שעוד לא הגיעה. הוא נמצא באחד משלושה מצבים: **pending** (ממתין), **fulfilled** (הצליח, יש ערך) או **rejected** (נכשל, יש שגיאה).

```
const promise = fetch('/api/projects');   // returns a Promise immediately

promise
  .then((response) => response.json())    // runs when it succeeds
  .then((data) => console.log(data))      // chained: gets the previous return value
  .catch((error) => console.error(error)) // runs if ANY step above failed
  .finally(() => console.log('done'));    // runs either way
```

async / await: לכתוב אסינכרוני כמו רגיל

מילת המפתח `await` "מחכה" ל-Promise ונותנת את הערך שלו, והקוד נראה כמו קוד סינכרוני רגיל. אפשר להשתמש בה רק בתוך פונקציה שמוגדרת `async`. פונקציית `async` מחזירה תמיד Promise.

```
async function loadProjects() {
  try {
    const response = await fetch('/api/projects');   // pause here until it arrives
    if (!response.ok) {
      throw new Error(`HTTP ${response.status}`);    // throw -> jumps to catch
    }
    const data = await response.json();
    return data;
  } catch (error) {
    console.error('Failed:', error.message);
    return [];
  } finally {
    console.log('finished either way');
  }
}

// An async function always returns a Promise, so from outside:
loadProjects().then((projects) => console.log(projects.length));

// Or, inside another async function:
const projects = await loadProjects();
```

שימו לב: אל תשכחו await

בלי `await` מקבלים את ה-Promise עצמו ולא את הערך שבתוכו: `const data = fetch(url)` נותן `Promise {<pending>}`. זו אחת הטעויות הנפוצות ביותר. סימן היכר: בקונסולה מופיע `Promise` במקום הנתונים.

כמה Promises במקביל

```
// Sequential: 2 seconds total (waits for each)
const a = await fetchUsers();
const b = await fetchProjects();

// Parallel: ~1 second total (both start at once)
const [users, projects] = await Promise.all([fetchUsers(), fetchProjects()]);

// Promise.all fails if ANY fails. To get every outcome:
const results = await Promise.allSettled([fetchUsers(), fetchProjects()]);
```

שליחת בקשות: fetch ו-axios

הדפדפן מגיע עם `fetch`. הפרויקט משתמש בספרייה **axios**, שעושה אותו דבר בנוחות רבה יותר: ממירה JSON אוטומטית, זורקת שגיאה על סטטוס 4xx/5xx ומאפשרת **interceptors**.

```
// fetch: you convert JSON yourself and check ok yourself
const res = await fetch('http://localhost:3000/api/tasks', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', Authorization: `Bearer ${token}` },
  body: JSON.stringify({ title: 'New task' }),
});
const task = await res.json();

// axios: shorter
import axios from 'axios';
const client = axios.create({ baseURL: 'http://localhost:3000/api' });
const { data } = await client.post('/tasks', { title: 'New task' });
//      ^ response body already parsed
```

```
await client.get('/projects', { params: { status: 'ACTIVE' } });   // ?status=ACTIVE
await client.patch(`/tasks/${id}`, { title: 'Edited' });
await client.delete(`/tasks/${id}`);
```

בפרויקט שלנו: קריאה של http.js

הקובץ `src/api/http.js` הוא דוגמה מצוינת שמכילה כמעט כל מה שלמדנו בפרק הזה. נעבור עליו בשלבים.

src/api/http.js (1) - creating the client and attaching the token

```
const baseURL = import.meta.env.VITE_API_URL ?? 'http://localhost:3000/api';

export const http = axios.create({ baseURL, withCredentials: true });

http.interceptors.request.use((config) => {
  const token = getAccessToken();
  if (token) {
    config.headers.set('Authorization', `Bearer ${token}`);
  }
  return config;
});
```

ה-**request interceptor** הוא פונקציה שרצה על **כל בקשה יוצאת** לפני שהיא נשלחת. כאן היא מוסיפה את הכותרת `Authorization` עם ה-token של המשתמש. כך אף מסך לא צריך לדאוג להעביר את ה-token בעצמו.
`import.meta.env` הוא הדרך של Vite לקרוא הגדרות סביבה (קובץ `.env`).

src/api/http.js (2) - refreshing an expired token, once

```
let refreshPromise = null;

async function refreshAccessToken() {
  if (!refreshPromise) {
    refreshPromise = refreshClient
      .post('/auth/refresh', {})
      .then((res) => {
        setTokens(res.data);
        return res.data.accessToken;
      })
      .catch(() => {
        clearTokens();
        return null;
      })
      .finally(() => {
        refreshPromise = null;
      });
  }
  return refreshPromise;
}
```

מה קורה כאן? ה-token של המשתמש פג תוקף אחרי 15 דקות. הפונקציה מבקשת token חדש. הטריק החכם: אם עשר בקשות נכשלו באותו רגע, כולן קוראות ל-`refreshAccessToken()` — אבל רק הראשונה יוצרת Promise חדש. השאר מקבלות את **אותו Promise** וממתינות לו, ולכן נשלחת בקשת רענון אחת בלבד. זו בעצם "נעילה" באמצעות משתנה חיצוני (closure).

src/api/http.js (3) - response interceptor with retry

```
http.interceptors.response.use(
  (res) => res,                       // success: pass through
  async (error) => {                  // failure: decide what to do
    const config = error.config;
    if (error.response?.status === 401 && config && !config._retried) {
      config._retried = true;
      const newToken = await refreshAccessToken();
      if (newToken) {
        config.headers.set('Authorization', `Bearer ${newToken}`);
        return http(config);          // repeat the original request
      }
      notifySessionExpired();
    }
    // ...
  },
);
```

הפונקציה השנייה ב-`use(onSuccess, onError)` מטפלת בשגיאות. אם השרת ענה `401` (לא מורשה), היא מרעננת את ה-token ו**שולחת שוב את אותה בקשה** — והמסך שביקש בכלל לא יודע שהיה כשל. זה מה שנקרא "שכבת תשתית": מסתירה מורכבות מכל שאר הקוד.

ה-Event Loop בקצרה

איך JavaScript מבצעת דברים "במקביל" בחוט אחד? יש **Call Stack** (מה שרץ עכשיו) ו**תור** של פעולות שהסתיימו וממתינות. כשהמחסנית מתרוקנת, JavaScript לוקחת את הפעולה הבאה מהתור. לכן `setTimeout(fn, 0)` מריץ את `fn` רק אחרי שהקוד הנוכחי סיים. מספיק לזכור: **קוד שאחרי await או בתוך then רץ מאוחר יותר**, אחרי שהקוד הסינכרוני הנוכחי נגמר.

נסו בעצמכם 6.1

נתבו מה מדפיס הקוד הבא ובאיזה סדר, ואז הריצו:

```
console.log('A');
setTimeout(() => console.log('B'), 0);
Promise.resolve().then(() => console.log('C'));
(async () => { console.log('D'); await null; console.log('E'); })();
console.log('F');
```

רמז: קוד אחרי `await` ו-`then` נכנס לתור "מיקרו-משימות", שמתפנה לפני `setTimeout`.

הדפדפן: DOM, אירועים ואחסון

React מסתירה את רוב העבודה הישירה עם הדף — אבל כדי להבין אותה צריך לדעת מה היא מסתירה.

ה-DOM

כשהדפדפן קורא HTML, הוא בונה **עץ** של אובייקטים שנקרא DOM (Document Object Model). JavaScript יכולה לקרוא ולשנות את העץ, והדפדפן מצייר מחדש.

index.html

```
<body>
  <div id="root"></div>
  <script type="module" src="/src/main.jsx"></script>
</body>
```

Manual DOM manipulation (what React does for you)

```
const root = document.getElementById('root');       // find an element
const button = document.querySelector('.save');      // by CSS selector

const p = document.createElement('p');              // create
p.textContent = 'Hello';                            // set text
p.classList.add('note');                            // CSS class
root.appendChild(p);                                // insert into the page

button.addEventListener('click', (event) => {       // react to a click
  console.log('clicked', event.target);
});
```

כתיבה ידנית כזו מסתבכת מהר: צריך לזכור לעדכן כל חלק במסך בכל שינוי בנתונים. כאן נכנסת React — נראה בחלק ב׳ שהיא מאפשרת לתאר "איך המסך אמור להיראות" ולהשאיר לה את העדכונים.

אירועים

אירוע (event) הוא דבר שקורה בדף: לחיצה, הקלדה, שליחת טופס, גלילה. מחברים פונקציה ("מאזין") שתרוץ כשהוא קורה. אובייקט ה-`event` מכיל פרטים.

אירוע	מתי	שימושי
click	לחיצה	event.target
input / change	שינוי בשדה קלט	event.target.value
submit	שליחת טופס	event.preventDefault()
keydown	לחיצת מקש	event.key

```
form.addEventListener('submit', (event) => {
  event.preventDefault();           // stop the browser's default: reloading the page
  const email = form.elements.email.value;
  signIn(email);
});
```

localStorage ו-sessionStorage

הדפדפן מאפשר לשמור מפתחות וערכים (מחרוזות בלבד) שנשארים אחרי רענון. `localStorage` נשמר "לתמיד", `sessionStorage` רק עד סגירת הלשונית.

```
localStorage.setItem('theme', 'dark');
localStorage.getItem('theme');           // 'dark'  (null if missing)
localStorage.removeItem('theme');

// Objects must be converted to text
localStorage.setItem('prefs', JSON.stringify({ lang: 'he' }));
const prefs = JSON.parse(localStorage.getItem('prefs') ?? '{}');
```

בפרויקט שלנו

ב-`AuthContext.jsx`: `localStorage.getItem('flowboard.session.activeRole') || 'EMPLOYEE'` — התפקיד הפעיל שהמשתמש בחר נשמר בין ביקורים. שימו לב לשם המפתח עם קידומת `flowboard.` — מונע התנגשות עם אתרים אחרים.

כלי הפיתוח של הדפדפן (F12)

לשונית	למה משתמשים
Console	הודעות שגיאה, `console.log`, והרצת קוד JavaScript ידנית.
Network	כל בקשות השרת: כתובת, סטטוס (200/401/500), גוף התשובה. **הכלי החשוב ביותר** כדי להבין מה השרת שולח.
Elements	ה-DOM הנוכחי וה-CSS של כל אלמנט.
Application	ה-`localStorage` והעוגיות (cookies).
Sources	הקוד עם נקודות עצירה (breakpoints) לדיבוג צעד אחר צעד.

טיפ: console.log חכם

```
console.log('projects', projects);          // label + value
console.log({ projects, filters });         // prints { projects: [...], filters: {...} } with
names
console.table(users);                       // an array of objects as a table
console.error('boom');                      // red
debugger;                                   // pauses here if DevTools is open
```

תרגילים וסיכום חלק א׳

לפני שעוברים ל-React, כדאי לוודא שהיסודות יושבים. הפתרונות בנספח ג׳.

תרגילי סיכום

8.1 — סטטיסטיקה של משימות

נתון:

```
const tasks = [
  { id: 1, title: 'Login page',  assignee: 'dana', priority: 'HIGH',   done: false },
  { id: 2, title: 'Fix bug 42',  assignee: 'omer', priority: 'URGENT', done: true  },
  { id: 3, title: 'Write docs',  assignee: 'dana', priority: 'LOW',    done: false },
  { id: 4, title: 'Refactor',    assignee: null,   priority: 'MEDIUM', done: false },
];
```

כתבו קוד שמחזיר: (א) שמות המשימות הפתוחות; (ב) כמה משימות יש לכל assignee (אובייקט כמו `{ dana: 2, omer: 1 }`, בלי המשימה ללא assignee); (ג) המשימה הראשונה בעדיפות URGENT או HIGH שעדיין פתוחה; (ד) האם כל המשימות של `dana` פתוחות.

8.2 — פונקציה אסינכרונית

כתבו `async function getUserName(id)` שמבצעת `fetch` ל-`https://jsonplaceholder.typicode.com/users/${id}` ומחזירה את שדה `name`. אם התשובה לא תקינה — היא תחזיר `'Unknown'`.

8.3 — קריאת קוד

קראו את הקוד הבא ותסבירו במילים מה הוא עושה, ולמה `refreshPromise` מוגדר מחוץ לפונקציה:

```
let refreshPromise = null;
async function refresh() {
  if (!refreshPromise) refreshPromise = doRefresh().finally(() => { refreshPromise = null; });
  return refreshPromise;
}
```

סיכום: מה אתם אמורים לדעת אחרי חלק א׳

- להבדיל בין `const` ל-`let`, להשתמש ב-`===`, ולקרוא `&&`, `||`, `??`, `?.` ו-`?:`.
- לקרוא ולכתוב arrow functions, כולל destructuring בפרמטרים.
- לפרק ולבנות אובייקטים ומערכים (`...`, destructuring) ולהשתמש בקונסולה וב-Network.
- להשתמש ב-`map`, `filter`, `find`, `some`, `every` ו-`reduce`.
- לקרוא שורות `import`/`export` ולהבין מאיפה מגיע כל דבר.
- להסביר מה Promise ומה `await` עושים, ולקרוא interceptors של axios.
- **בדיקה עצמית:** אם אתם מסוגלים להסביר כל שורה ב-`src/lib/format.js` וב-`permissions.js` — אתם מוכנים לחלק ב׳.

חלק ב׳

React

ספריית ה-UI שמאחורי כל מסך בפרויקט, וחבילות ההמשך שלה: Router לניווט, TanStack Query לנתוני שרת ו-MUI לרכיבי ממשק.

חלק ב׳ · פרק 9

קומפוננטות, JSX ו-props

הרעיון המרכזי של React במשפט אחד: **ממשק הוא פונקציה של נתונים**. נותנים לה נתונים, והיא מחזירה תיאור של איך המסך נראה.

מה זה React ולמה משתמשים בו

בפרק 7 ראינו שעדכון ידני של הדף מסתבך. React פותרת את זה בגישה **דקלרטיבית**: במקום להורות לדפדפן "מצא את האלמנט, שנה את הטקסט שלו, הוסף מחלקה", אתם מתארים **איך המסך אמור להיראות בהינתן הנתונים הנוכחיים**. כשהנתונים משתנים, React משווה בין התיאור הקודם לחדש ומעדכנת בדפדפן רק את ההבדלים.

Imperative (manual DOM) vs declarative (React)

```
// Imperative: you manage every change yourself
counterEl.textContent = count;
if (count > 5) warningEl.classList.add('visible'); else warningEl.classList.remove('visible');

// Declarative: describe the result for the current data
<div>
  {count}
  {count > 5 && <p className="warning">Too many!</p>}
</div>
```

קומפוננטה = פונקציה שמחזירה JSX

קומפוננטה (רכיב) היא פונקציית JavaScript רגילה ששמה מתחיל **באות גדולה** ומחזירה תיאור של ממשק. את התיאור כותבים ב-**JSX**: תחביר שנראה כמו HTML בתוך JavaScript.

```jsx
function Greeting() {
  return <h1>Hello, world</h1>;
}

// Using it - like a custom HTML tag
function App() {
  return (
    <div>
      <Greeting />
      <Greeting />
    </div>
  );
}
```

שימו לב: האות הגדולה חובה

`<greeting />` React תחשוב שזה תג HTML רגיל. `<Greeting />` היא קומפוננטה. בקוד תמיד: קומפוננטות באות גדולה (`PascalCase`), פונקציות ומשתנים רגילים באות קטנה (`camelCase`).

ה-JSX הוא רק "סוכר תחבירי". מאחורי הקלעים הוא מתורגם לקריאות פונקציה. אתם לא צריכים לכתוב את זה — אבל זה עוזר להבין למה מותר מה שמותר:

```jsx
<button className="save" onClick={save}>Save</button>

// is compiled to (roughly):
createElement('button', { className: 'save', onClick: save }, 'Save')
```

חוקי ה-JSX

כלל	דוגמה
אלמנט שורש אחד בלבד (או Fragment)	<> <A /> <B /> </>
כל תג נסגר, גם בודד	
`className` במקום `class`	
`htmlFor` במקום `for`	<label htmlFor="email">
מאפיינים ב-camelCase	onClick, onChange, tabIndex
ביטוי JavaScript בתוך `{ }`	{user.name}
`style` מקבל אובייקט	style={{ color: 'red' }}
הערה בתוך JSX	{/* comment */}

```jsx
function Profile() {
  const user = { name: 'Dana', age: 30, avatar: '/dana.png' };
  const isAdult = user.age >= 18;

  return (
    <>                                          {/* Fragment: a wrapper that renders nothing */}
      <img src={user.avatar} alt={user.name} />   {/* attribute = expression in braces */}
      <h2 style={{ color: 'navy', marginTop: 8 }}>  {/* outer {} = JS, inner {} = the object */}
        {user.name} ({user.age})                  {/* text + expressions */}
      </h2>
      <p className="status">{isAdult ? 'Adult' : 'Minor'}</p>
      <p>{user.age + 1} next year, {user.name.toUpperCase()}</p>
```

```
  );
}
```

שימו לב: בתוך `{ }` מותרים רק ביטויים

אפשר לשים בתוך הסוגריים המסולסלים כל דבר שמחזיר ערך (`a + b`, קריאה לפונקציה, טרנרי). אי אפשר לשים פקודות כמו `if` או `for` — לשם כך יש טכניקות שנראה מיד: טרנרי, `&&` ו-`map`.

Props: העברת נתונים לקומפוננטה

Props (מהמילה properties) הם ה"פרמטרים" של קומפוננטה. כמו מאפייני HTML, אבל יכולים להיות כל ערך: מספר, אובייקט, פונקציה ואפילו קומפוננטה אחרת. React אוספת את כולם לאובייקט אחד ומעבירה אותו כארגומנט יחיד — ולכן כל כך הרבה קוד נראה כמו destructuring בחתימה (פרק 3).

```
function Welcome({ name, role = 'guest' }) {        // props destructured, role has a default
  return <p>Hello {name} ({role})</p>;
}

<Welcome name="Dana" role="admin" />       // strings: quotes
<Welcome name="Dana" />                    // role -> 'guest'

// Non-string values go in braces:
<ProjectCard project={p} max={4} showRisk={true} onOpen={() => open(p.id)} />
<Modal open />                             // a prop without a value = true
```

שלושה כללים על props

- **הם זורמים למטה בלבד:** הורה נותן לילד, לא להפך. כדי שילד "יעדכן" את ההורה, ההורה מעביר לו **פונקציה** (`onSave`) והילד קורא לה.
- **הם לקריאה בלבד:** קומפוננטה לא משנה את ה-props שלה.
- **שינוי prop גורם לרינדור מחדש** של הקומפוננטה.

children: תוכן בין התגים

מה שכותבים בין תג הפתיחה לסגירה של קומפוננטה מגיע אליה כ-prop בשם `children`. כך בונים קומפוננטות "מסגרת".

```
function Card({ title, children }) {
  return (
    <div className="card">
      <h3>{title}</h3>
      <div className="card-body">{children}</div>
    </div>
  );
}

<Card title="Tasks">
  <p>You have 3 tasks.</p>
  <button>Open</button>
</Card>
// children = the <p> and the <button>
```

בפרויקט שלנו

ב-`main.jsx` כל ה-Providers (`ThemeProvider`, `AuthProvider`...) הם קומפוננטות שעוטפות `children`.
וב-`AuthProvider` עצמו: `function AuthProvider({ children })` ובסוף `<AuthContext.Provider value={value}>{children}</AuthContext.Provider>`.

רינדור מותנה

שלוש דרכים, ותראו את כולן:

```
// 1. Ternary - either/or
{isLoading ? <Spinner /> : <List items={items} />}

// 2. && - show or nothing
{isManager && <Button>Add Task</Button>}

// 3. Early return - the component decides not to render the usual thing
function Page() {
  if (isLoading) return <LoadingState label="Loading…" />;
  if (!project) return <EmptyState title="Not found" />;
  return <Board project={project} />;
}

// Chained ternary (an "else if" ladder)
{items.length === 0 ? <Empty /> : items.length > 100 ? <Big /> : <Normal />}
```

שימו לב: המלכודת של `0 &&`

`{count && <Badge />}` כש-`count` הוא `0` ירנדר את המספר **0** במסך (כי `0` הוא falsy ו-`&&` מחזיר אותו).
הפתרון: `{count > 0 && <Badge />}`.

בפרויקט שלנו: שרשרת טרנרי ב-MyProjectsPage

```
{(projects ?? []).length === 0 ? (
  <EmptyState title="You're not part of any project yet" ... />
) : filtered.length === 0 ? (
  <NoSearchResultsState onClear={() => { setSearch(''); setStatus('ALL'); }} />
) : (
  <Grid container spacing={2}> ... </Grid>
)}
```

קראו: "אם אין פרויקטים בכלל — הצג מצב ריק. אחרת, אם המסנן לא מצא כלום — הצג 'אין תוצאות'. אחרת — הצג את רשת הפרויקטים." שלושה מצבים, ביטוי אחד.

רשימות ו-`key`

כדי להציג רשימה, ממירים מערך נתונים למערך אלמנטים עם `map` — React יודעת לרנדר מערך של אלמנטים. כל איבר חייב `key` ייחודי ויציב, כדי ש-React תזהה איזה איבר הוא איזה כשהרשימה משתנה.

```jsx
<ul>
  {tasks.map((task) => (
    <li key={task.id}>
      {task.title} {task.done && '✓'}
    </li>
  ))}
</ul>
```

שימו לב

- ה-`key` חייב להיות ה-**id של הנתון**, לא האינדקס (`i`) כשהרשימה יכולה להשתנות או להתמיין. אינדקס גורם לבאגים בטפסים ובאנימציות.
- ה-`key` נכתב על האלמנט **החיצוני ביותר** שמחזירה ה-`map`, והוא לא מגיע לקומפוננטה כ-prop.

בפרויקט שלנו

MyProjectsPage.jsx

```jsx
{filtered.map((p) => (
  <Grid key={p.id} size={{ xs: 12, sm: 6, lg: 4 }}>
    <ProjectCard
      project={p}
      manager={userMap.get(p.managerId)}
      members={p.memberIds.map((id) => userMap.get(id)).filter(Boolean)}
    />
  </Grid>
))}
```

לכל פרויקט `p` נוצר `Grid` עם כרטיס. שימו לב לתבנית `.map(...).filter(Boolean)`: מחליפים מזהים באובייקטים, ואז זורקים את אלה שלא נמצאו (`undefined` הוא falsy).

נסו בעצמכם 9.1

כתבו קומפוננטה `UserList` שמקבלת prop `users` (מערך של `{id, name, active}`) ומציגה רשימה: משתמש פעיל עם ✓ ולא פעיל באפור (`style`). אם הרשימה ריקה — "No users". רשמו גם את הקריאה: `<UserList users={[...]} />`.

state ואירועים

משתנים רגילים לא גורמים למסך להתעדכן. **state** כן. זה המנגנון שהופך דף סטטי לאפליקציה חיה.

למה לא מספיק משתנה רגיל

```
function Counter() {
  let count = 0;                        // plain variable

  return (
    <button onClick={() => { count = count + 1; console.log(count); }}>
      Clicked {count} times
    </button>
  );
}
// console shows 1, 2, 3... but the screen still says "Clicked 0 times"
```

שתי סיבות: (1) React לא יודעת ש-`count` השתנה ולכן לא מרנדרת מחדש; (2) גם אם הייתה מרנדרת, הפונקציה `Counter` הייתה רצה מההתחלה ו-`count` היה חוזר ל-`0`. צריך מקום ש-React **זוכרת** בין רינדורים ושגם **מודיע לה** כשמשהו משתנה.

useState

```
import { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);   // [current value, function to change it]

  return (
    <button onClick={() => setCount(count + 1)}>
      Clicked {count} times
    </button>
  );
}
```

נפרק שורה אחר שורה:

- `useState(0)` — "אני צריך תא זיכרון, והערך ההתחלתי שלו הוא 0". מחזיר **מערך בן שני איברים**.
- `const [count, setCount] = ...` — destructuring של מערך (פרק 4): הראשון הוא הערך הנוכחי, השני פונקציה לעדכון. השמות בחירה שלכם; המוסכמה: `x` ו-`setX`.
- `setCount(count + 1)` — לא משנה את `count` ישירות! מבקשת מ-React: "הערך החדש הוא זה, תרנדרי מחדש". בפעם הבאה שהפונקציה רצה, `useState` תחזיר את הערך המעודכן.

1. לחיצה על הכפתור
2. `setCount(1)` — מבקשים מ-React לעדכן
3. React מתזמנת רינדור חדש
4. הפונקציה `Counter()` רצה שוב, ו-`useState` מחזירה 1
5. המסך מתעדכן

מודל מנטלי: "הפונקציה רצה מחדש בכל שינוי"

כל פעם ש-state או props משתנים, **כל גוף הקומפוננטה רץ שוב מההתחלה**. כל ריצה היא "צילום" של המסך עם הערכים של אותו רגע. אין "עדכון" — יש ריצה חדשה. זה למה משתנים רגילים בתוכה מאופסים, ורק `useState` שורד.

עדכון שתלוי בערך הקודם

```
setCount(count + 1);
setCount(count + 1);
setCount(count + 1);   // still only +1 total! all three read the same "snapshot" of count

// The functional form receives the latest value:
setCount((c) => c + 1);
setCount((c) => c + 1);
setCount((c) => c + 1);   // +3
```

כשהערך החדש תלוי בקודם, השתמשו בצורת הפונקציה (`(prev) => ...`).

state של אובייקטים ומערכים: לא משנים — מחליפים

זה הכלל הכי חשוב בפרק. React מזהה שינוי לפי **השוואת הפניות** (`===`). אם משנים אובייקט קיים (`user.name = 'x'`) וקוראים ל-`setUser(user)`, ההפניה זהה — React חושבת שלא קרה כלום ולא מרנדרת. לכן תמיד יוצרים **עותק חדש** עם spread (פרק 4).

```
const [user, setUser] = useState({ name: 'Dana', age: 30, address: { city: 'Haifa' } });
const [tasks, setTasks] = useState([]);

// WRONG - mutates the existing object; React may not re-render
user.age = 31;
setUser(user);

// RIGHT - a new object
setUser({ ...user, age: 31 });
setUser((u) => ({ ...u, address: { ...u.address, city: 'Tel Aviv' } }));   // nested

// Arrays
setTasks([...tasks, newTask]);                          // add
setTasks(tasks.filter((t) => t.id !== id));             // remove
setTasks(tasks.map((t) => t.id === id ? { ...t, done: true } : t));   // update one
```

בפרויקט שלנו

src/auth/AuthContext.jsx

```
const setActiveRole = async (role) => {
  if (!user?.roles.includes(role)) return;
  setActiveRoleState(role);
  setUser((current) => current ? { ...current, defaultRole: role } : current);
  await updateUserProfile(user.id, { defaultRole: role });
};
```

שלוש שורות מהפרק: `setUser` בצורת פונקציה (`current => ...`), עם spread (`{ ...current, defaultRole: role }`) ועם הגנה על מקרה של `null`.

אירועים ב-React

מחברים handler לאלמנט עם מאפיין בשם `onXxx` (camelCase), ומעבירים **פונקציה** — לא קריאה לפונקציה.

```
<button onClick={handleSave}>Save</button>          // pass the function
<button onClick={handleSave()}>Save</button>        // WRONG: calls it during render!
<button onClick={() => handleDelete(task.id)}>Delete</button>   // need an argument -> wrap in an arrow

<input onChange={(e) => setName(e.target.value)} />
<form onSubmit={(e) => { e.preventDefault(); save(); }}>
<input onKeyDown={(e) => { if (e.key === 'Enter') submit(); }} />
```

שימו לב: `onClick={fn()}` לעומת `onClick={() => fn()}`

הראשון מריץ את `fn` מיד בזמן הרינדור ומעביר את התוצאה. השני יוצר פונקציה שתרוץ בלחיצה. כשצריך להעביר ארגומנט — עוטפים ב-arrow.

Controlled inputs: טפסים

ב-React שדה קלט "מבוקר" (controlled): ה-state הוא מקור האמת, והשדה מציג אותו. כל הקלדה מפעילה `onChange` שמעדכן את ה-state, שמרנדר את השדה מחדש עם הערך החדש.

```
function SearchBox() {
  const [query, setQuery] = useState('');

  return (
    <input
      value={query}                              // the field shows the state
      onChange={(e) => setQuery(e.target.value)} // typing updates the state
      placeholder="Search…"
    />
  );
}

// A form with several fields - one state object
function SignIn() {
  const [form, setForm] = useState({ email: '', password: '' });
  const [error, setError] = useState('');

  const update = (field) => (e) => setForm({ ...form, [field]: e.target.value });

  const handleSubmit = async (e) => {
    e.preventDefault();                          // no page reload
    try { await signIn(form); }
    catch (err) { setError(err.message); }
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={form.email} onChange={update('email')} />
      <input type="password" value={form.password} onChange={update('password')} />
      {error && <p role="alert">{error}</p>}
      <button type="submit">Sign in</button>
    </form>
  );
}
```

בפרויקט שלנו: שדה חיפוש

MyProjectsPage.jsx

```
const [search, setSearch] = useState('');
...
<TextField
  size="small"
  placeholder="Search projects…"
  value={search}
  onChange={(e) => setSearch(e.target.value)}
/>
```

זה בדיוק אותו דפוס, עם `TextField` של MUI במקום `input`. ה-state `search` נשלח גם לשרת כמסנן, ולכן כל הקלדה מביאה רשימה מסוננת.

לא לשמור ב-state מה שאפשר לחשב

טעות נפוצה: לשמור ב-state ערך שנגזר מ-state אחר. במקום זאת מחשבים אותו בכל רינדור.

```
const [tasks, setTasks] = useState([]);
const [openCount, setOpenCount] = useState(0);    // WRONG: must be kept in sync by hand

const openCount = tasks.filter((t) => !t.done).length;   // RIGHT: derived on every render
```

אתחול עצלן ו"הרמת state"

```
// Lazy initial value: the function runs only on the first render
const [role, setRole] = useState(() => localStorage.getItem('role') || 'EMPLOYEE');
```

בפרויקט שלנו

זו בדיוק השורה ב-`AuthContext.jsx`: `useState(() => localStorage.getItem(ROLE_KEY) || 'EMPLOYEE')`. ללא הפונקציה, `localStorage` היה נקרא בכל רינדור לחינם.

Lifting state up: כששני רכיבים צריכים את אותו נתון, מעבירים את ה-state להורה המשותף שלהם, ומעבירים לילדים את הערך (prop) ואת פונקציית העדכון (prop). ב-`MyProjectsPage` ה-state של `search` ו-`status` יושב בעמוד — כי גם שדה החיפוש, גם הצ׳יפים וגם רשימת התוצאות תלויים בו.

נסו בעצמכם 10.1

בנו קומפוננטת `TodoApp`: שדה טקסט וכפתור "Add" שמוסיף משימה למערך ב-state; רשימה של המשימות עם כפתור "✓" (הופך `done`) וכפתור "×" (מוחק); ומתחת — "open 3" (מחושב, לא ב-state). אפשר לבדוק ב-`https://react.new`.

Effects והוקים נוספים

קומפוננטה צריכה לפעמים לעשות דבר שלא קשור לציור: לפתוח חיבור, להירשם לאירוע, לקרוא מהשרת. בשביל זה יש useEffect.

מהם "הוקים" (Hooks)

הוק הוא פונקציה שמתחילה ב-`use` ומאפשרת לקומפוננטה להשתמש ביכולות של React (state, מחזור חיים, הקשר). שני כללים מחייבים:

1. קוראים להוקים **רק ברמה העליונה** של קומפוננטה — לא בתוך `if`, לולאה או פונקציה פנימית.
2. קוראים להם **רק** מתוך קומפוננטה או מתוך הוק אחר (`use...`).

הסיבה: React מזהה כל הוק לפי **הסדר** שבו הוא נקרא בכל רינדור, ולכן הסדר חייב להיות זהה תמיד.

useEffect

Effect הוא קוד שרץ **אחרי** שהקומפוננטה צוירה במסך. משתמשים בו לדברים "מחוץ" ל-React: חיבור לשרת, טיימרים, האזנה לאירועי דפדפן, עדכון `document.title`.

```
useEffect(() => {
  // 1. the effect: runs after render
  document.title = `${count} tasks`;

  return () => {
    // 2. optional cleanup: runs before the next effect, and when the component disappears
  };
}, [count]);   // 3. dependency array
```

מערך התלויות: מתי ה-effect רץ

כתיבה	מתי רץ
useEffect(fn)	אחרי **כל** רינדור (כמעט אף פעם לא רוצים).
useEffect(fn, [])	פעם אחת — כשהקומפוננטה מופיעה (mount). ה-cleanup — כשהיא נעלמת.
useEffect(fn, [a, b])	אחרי הרינדור הראשון, ואחר כך בכל פעם ש-`a` או `b` השתנו.

כלל אצבע

כל ערך מהקומפוננטה (state, prop, פונקציה) שה-effect **משתמש בו** חייב להופיע במערך התלויות. הכלי `eslint-plugin-react-hooks` (שמוגדר בפרויקט) מתריע אם שכחתם.

Cleanup: לנקות אחרי עצמכם

כל דבר שה-effect "פותח" — האזנה, טיימר, חיבור — צריך להיסגר בפונקציית ה-cleanup, אחרת נצבור דליפות (כמה מאזינים במקביל, טיימרים שממשיכים לרוץ).

```
useEffect(() => {
  const onResize = () => setWidth(window.innerWidth);
  window.addEventListener('resize', onResize);            // open

  return () => window.removeEventListener('resize', onResize);   // close
}, []);

useEffect(() => {
  const id = setInterval(() => setSeconds((s) => s + 1), 1000);
  return () => clearInterval(id);
}, []);
```

בפרויקט שלנו: התראות בזמן אמת

זה הקובץ `src/lib/useNotificationsSocket.js`. קראו אותו עם מה שלמדתם:

```
export function useNotificationsSocket() {
  const { user } = useAuth();
  const queryClient = useQueryClient();

  useEffect(() => {
    if (!user) return;                                    // not signed in: do nothing
    const connection = createNotificationsConnection();   // open a connection to the server
    const queryKey = ['notifications', user.id];

    const handleNew = (raw) => {
      const notification = mapNotification(raw);
      queryClient.setQueryData(queryKey, (prev) => (prev ? [notification, ...prev] :
[notification]));
    };
    connection.on('notification:new', handleNew);         // subscribe to server messages
    connection.start().catch(() => {});

    return () => {                                        // CLEANUP
      connection.off('notification:new', handleNew);
      connection.stop().catch(() => {});                  // close the connection
    };
  }, [user, queryClient]);                                // re-run if the user changes
}
```

- ה-effect פותח חיבור ל-SignalR ומאזין להודעות. כשמגיעה התראה חדשה — מוסיפים אותה לראש הרשימה במטמון (פרק 14).
- ה-cleanup סוגר את החיבור. הוא רץ כשהמשתמש מתנתק (ה-`user` משתנה) או כשהרכיב נעלם.
- `return` הראשון (`if (!user) return;`) הוא יציאה מוקדמת מה-effect — בלי cleanup כי לא נפתח דבר.

קריאת נתונים ב-useEffect — ולמה לא עושים את זה ידנית

אפשר להביא נתונים מהשרת ב-effect, אבל זה דורש טיפול ב-loading, שגיאות, ביטול ו"race conditions" (תשובה ישנה שמגיעה אחרי חדשה). דפוס נפוץ הוא דגל `cancelled`:

```
useEffect(() => {
  let cancelled = false;                    // closure variable

  async function load() {
    const data = await fetchProject(projectId);
    if (!cancelled) setProject(data);       // ignore the answer if we've moved on
  }
  load();

  return () => { cancelled = true; };       // cleanup marks the effect as stale
}, [projectId]);
```

בפרויקט שלנו

בדיוק הדפוס הזה ב-`AuthContext.jsx`, בפונקציה `bootstrap`: `let cancelled = false; ... if (cancelled) return; ... return () => { cancelled = true; }`. בשאר המסכים הפרויקט נמנע מכל זה באמצעות TanStack Query (פרק 14), שעושה את כל הטיפול הזה בשבילנו.

StrictMode: למה ה-effect רץ פעמיים?

ב-`main.jsx` האפליקציה עטופה ב-`<StrictMode>`. **במצב פיתוח בלבד** הוא מריץ כל effect פעמיים (mount → cleanup → mount) כדי לחשוף effects ללא cleanup נכון. אם אתם רואים `console.log` פעמיים בפיתוח — זו הסיבה, ובפרודקשן זה לא קורה. אם האפליקציה מתנהגת אחרת בגלל זה — חסר cleanup.

useRef

`useRef` נותן "קופסה" `{ current: ... }` ששורדת בין רינדורים, אבל **שינוי בה לא גורם לרינדור**. יש לה שני שימושים: גישה לאלמנט DOM, ושמירת ערך "טכני" (טיימר, ערך קודם).

```
function FocusInput() {
  const inputRef = useRef(null);

  useEffect(() => { inputRef.current.focus(); }, []);      // focus on mount

  return <input ref={inputRef} />;    // React sets inputRef.current to the DOM element
}
```

בפרויקט תראו את זה ב-`KanbanColumn`: `const { setNodeRef } = useDroppable(...)` ואז `<Box ref={setNodeRef}>` — ספריית dnd-kit מקבלת את אלמנט ה-DOM כדי לדעת איפה אפשר לשחרר.

useCallback-ו useMemo

`useMemo` "זוכר" תוצאת חישוב ומחשב אותה מחדש רק כשהתלויות שלה משתנות. `useCallback` אותו דבר עבור פונקציה. שניהם **אופטימיזציה**, לא תכונה — קוד עובד גם בלעדיהם.

```
const visible = useMemo(
  () => tasks.filter((t) => t.title.includes(query)),   // recomputed only when tasks/query change
  [tasks, query],
);

const handleSave = useCallback(() => save(form), [form]);   // same function identity until form
changes
```

בפרויקט שלנו

ב-`KanbanBoardPage` ה-`useMemo` משמש לחישובים על רשימות: `filteredTasks`, `userMap`, `summary`.
כשקוראים אותו — קראו רק את הפונקציה בפנים כאילו אין עטיפה, והמערך שבסוף הוא "מתי לחשב מחדש".

מתי צריך?

כמעט תמיד לא. משתמשים בהם כשהחישוב באמת כבד, או כשמעבירים אובייקט/פונקציה לילד עטוף ב-`React.memo` או כתלות ב-`useEffect` (כי יצירה מחדש בכל רינדור תגרום להפעלה מחדש). כשאתם קוראים קוד — הבינו "זה לחסוך חישוב" ועברו הלאה.

נסו בעצמכם 11.1

כתבו הוק `useWindowWidth()` שמחזיר את רוחב החלון הנוכחי ומתעדכן כשמשנים גודל. ודאו שיש cleanup. אחר כך השתמשו בו בקומפוננטה שמציגה "Mobile" כשהרוחב קטן מ-600.

Context והוקים מותאמים אישית

איך נותנים מידע כמו "מי המשתמש המחובר" לכל רכיב באפליקציה בלי להעביר אותו דרך עשרה רמות של props.

הבעיה: Props drilling

נניח שרכיב בעומק העץ צריך את שם המשתמש. בלי Context צריך להעביר אותו כ-prop דרך כל רכיב בדרך, גם אלה שלא משתמשים בו. **Context** הוא "ערוץ" שמאפשר לכל רכיב מתחת ל-Provider לשלוף ערך ישירות.

App ◀ Layout ◀ Sidebar ◀ UserMenu ◀ Avatar :בלי Context — ה-user מועבר כ-prop דרך כולם

App קורא אותו ישירות ◀····· (user מחזיק) Context: AuthProvider :עם

שלושה שלבים

```
import { createContext, useContext, useState } from 'react';

// 1. Create the context
const ThemeContext = createContext('light');            // default value

// 2. Provide a value to everything inside
function ThemeProvider({ children }) {
  const [theme, setTheme] = useState('light');
  const value = { theme, toggle: () => setTheme((t) => (t === 'light' ? 'dark' : 'light')) };
  return <ThemeContext.Provider value={value}>{children}</ThemeContext.Provider>;
}

// 3. Consume it anywhere below
function ThemeButton() {
  const { theme, toggle } = useContext(ThemeContext);
  return <button onClick={toggle}>Now: {theme}</button>;
}

<ThemeProvider>
  <Page>
    <ThemeButton />       {/* no props passed */}
  </Page>
</ThemeProvider>
```

כשה-`value` של ה-Provider משתנה, כל הרכיבים שקוראים את ה-Context מתרנדרים מחדש.

AuthContext: הדוגמה האמיתית

בפרויקט שלנו: src/auth/AuthContext.jsx

זה הקובץ שמנהל "מי מחובר". שימו לב לארבעה חלקים:

src/auth/AuthContext.jsx (condensed)

```
const AuthContext = createContext(undefined);                    // (1) the channel

export function AuthProvider({ children }) {                     // (2) the provider
```

```
  const [user, setUser] = useState(null);
  const [organization, setOrganization] = useState(null);
  const [isLoading, setIsLoading] = useState(true);
  const [activeRole, setActiveRoleState] = useState(() => localStorage.getItem(ROLE_KEY) ||
'EMPLOYEE');

  useEffect(() => { /* on first load: fetch the current session from the server */ }, []);

  const signOut = () => { setUser(null); setOrganization(null); void apiSignOut(); };

  const value = {                                                    // what consumers get
    isAuthenticated: !!user,
    isLoading, user, organization, activeRole,
    availableRoles: user?.roles ?? [],
    setActiveRole, signInWithResult, signOut, refreshUser,
  };

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export function useAuth() {                                          // (3) the convenience hook
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth must be used within AuthProvider');   // (4) a safety check
  return ctx;
}
```

1. **ה-Provider מחזיק את ה-state** (משתמש, ארגון, תפקיד פעיל) ואת הפעולות (`signOut`).
2. **`useAuth()`** עוטף את `useContext(AuthContext)`, כדי שכל קומפוננטה תכתוב פשוט `const { user } = useAuth()`.
3. **הבדיקה** `if (!ctx) throw` נותנת הודעת שגיאה ברורה אם ניסו להשתמש בו מחוץ ל-Provider (כי ערך ברירת המחדל הוא `undefined`).

ולכן `main.jsx` עוטף את `App` ב-`AuthProvider`: זה מה שמאפשר לכל מסך לקרוא `useAuth()`.

Custom Hooks: הוקים משלכם

הוק מותאם אישית הוא פשוט פונקציה ששמה מתחיל ב-`use` וקוראת להוקים אחרים. מטרתו: לארוז לוגיקה חוזרת, כדי שרכיבים יישארו קצרים וקריאים. הוא לא מחזיר JSX אלא נתונים או פונקציות.

```
function useLocalStorage(key, initial) {
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    return saved ? JSON.parse(saved) : initial;
  });

  useEffect(() => {
    localStorage.setItem(key, JSON.stringify(value));
  }, [key, value]);

  return [value, setValue];          // same shape as useState
}

// Any component:
const [theme, setTheme] = useLocalStorage('theme', 'light');
```

בפרויקט שלנו: אוסף הוקים ב-queries.js

הקובץ `src/lib/queries.js` הוא רשימה ארוכה של הוקים מותאמים: `useProjects`, `useBoard`, `useTasks`, `useRoles` ועוד. כל אחד עוטף `useQuery` של TanStack Query עם המפתח והפונקציה הנכונים. כך במסך כותבים שורה אחת: `const { data: projects } = useProjects()` ולא צריכים לדעת איך הנתונים מגיעים.

איך לקרוא הוק מותאם

שלוש שאלות: (1) **מה הוא מקבל** (הפרמטרים)? (2) **מה הוא מחזיר** (ה-`return`)? (3) **אילו הוקים הוא משתמש בהם** בפנים? זהו — ככה מבינים אותו בלי לרדת לכל פרט.

נסו בעצמכם 12.1

בנו `CartContext` עם Provider שמחזיק מערך `items` ופעולות `add(item)` ו-`remove(id)`, והוק `useCart()`. עם בדיקת "מחוץ ל-Provider". הציגו בשתי קומפוננטות נפרדות: כפתור "Add" ומונה "Items: N".

ניווט עם React Router

אפליקציית SPA טוענת דף אחד, אבל המשתמש מצפה לכתובות שונות לכל מסך, לכפתור "אחורה" ולקישורים שאפשר לשתף. React Router מספק את זה.

הרעיון

ה-Router מסתכל על הכתובת (URL) בדפדפן ומחליט **איזו קומפוננטה להציג**. כשהמשתמש עובר לכתובת אחרת — הוא מחליף קומפוננטה בלי לטעון דף מחדש ובלי לפגוש את השרת.

רכיב / הוק	תפקיד
<BrowserRouter>	עוטף את האפליקציה ומפעיל את הניווט. (ב-`main.jsx`.)
<Routes> / <Route>	טבלת "כתובת ← קומפוננטה".
<Link to> / <NavLink>	קישור בלי טעינת דף. `NavLink` יודע להדגיש קישור פעיל.
useNavigate()	ניווט מתוך קוד (אחרי לחיצה או שמירה).
useParams()	קריאת חלקים דינמיים מהכתובת (`:projectId`).
<Navigate to>	הפניה אוטומטית בזמן רינדור.
<Outlet>	"חור" בתוך מסגרת שבו מוצג הנתיב הילד.
useLocation()	הכתובת הנוכחית (`pathname`, `search`, `state`).

Routes ו-Route

```
<Routes>
  <Route path="/sign-in" element={<SignInPage />} />
  <Route path="/projects" element={<MyProjectsPage />} />
  <Route path="/projects/:projectId" element={<ProjectOverviewPage />} />   {/* :projectId = dynamic
*/}
  <Route path="/" element={<Navigate to="/dashboard" replace />} />         {/* redirect */}
  <Route path="*" element={<NotFoundPage />} />                              {/* anything else */}
</Routes>
```

הצורה `:projectId` היא **פרמטר**: הכתובת `/projects/p7` תתאים, והערך `'p7'` זמין לקומפוננטה:

```
import { useParams, useNavigate, Link } from 'react-router-dom';

function ProjectOverviewPage() {
  const { projectId } = useParams();           // 'p7'
  const navigate = useNavigate();
  const { data: project } = useProject(projectId);

  return (
    <>
      <Link to={`/projects/${projectId}/board`}>Open board</Link>
      <button onClick={() => navigate('/projects')}>Back to list</button>
      <button onClick={() => navigate(-1)}>Back (browser history)</button>

  );
}
```

בפרויקט שלנו

ב-`ProjectCard.jsx`: `onClick={() => navigate(`/projects/${project.id}`)}`. לחיצה על כרטיס פרויקט מנווטת למסך שלו. ב-`App.jsx` מוגדר `path="/projects/:projectId"` ו-`path="/projects/:projectId/board"` — ושני המסכים קוראים את `projectId` עם `useParams`.

Layout routes ו-Outlet

לכל מסך יש אותה מסגרת: תפריט צד וסרגל עליון. במקום לחזור עליהם בכל מסך, מגדירים **Route בלי path** ש"עוטף" נתיבים ילדים. הקומפוננטה שלו מציבה `<Outlet />` במקום שבו הילד יוצג.

Simplified from App.jsx

```
<Routes>
  <Route path="/sign-in" element={<SignInPage />} />

  <Route element={<ProtectedRoute />}>          {/* layer 1: must be logged in */}
    <Route element={<AppShell />}>              {/* layer 2: sidebar + topbar */}
      <Route path="/dashboard" element={<PersonalDashboardPage />} />
      <Route path="/projects"  element={<MyProjectsPage />} />
      ...
    </Route>
  </Route>
</Routes>

// AppShell.jsx renders the frame and puts the current page in <Outlet />
function AppShell() {
  return (
    <Box>
      <Sidebar />
      <Topbar />
      <main><Outlet /></main>
    </Box>
  );
}
```

כשהכתובת היא `/projects`: React Router מפעיל `ProtectedRoute` ← `AppShell` ← `MyProjectsPage`, כל אחד בתוך ה-`Outlet` של קודמו.

הגנה על נתיבים

בפרויקט שלנו: src/auth/ProtectedRoute.jsx

```
export function ProtectedRoute() {
  const { isAuthenticated, isLoading } = useAuth();
  const location = useLocation();
  if (isLoading) {
    return <Box ...><CircularProgress /></Box>;              // still checking the session
  }
  if (!isAuthenticated) {
    return <Navigate to="/sign-in" replace state={{ from: location.pathname }} />;
  }
  return <Outlet />;                                         // ok: show the child route
}

export function RequireRoleLevel({ minLevel, children }) {
  const { activeRole } = useAuth();
  if (ROLE_LEVEL[activeRole] < minLevel) {
    return <Navigate to="/403" replace />;
  }
  return <>{children};
}
```

- `ProtectedRoute` הוא "שומר סף" בשלושה מצבים: טוען, לא מחובר (הפניה ל-`/sign-in`), או מחובר (מציג את הילדים דרך `Outlet`).
- `replace` מחליף את הכתובת בהיסטוריה, כדי שכפתור "אחורה" לא יחזיר לדף שמפנה מחדש.
- `state={{ from: ... }}` מעביר מידע לדף היעד (כדי לחזור לדף המקורי אחרי התחברות), וקוראים אותו עם `useLocation().state`.
- `RequireRoleLevel` עוטף מסכי ניהול ב-`App.jsx` כך:

`quireRoleLevel minLevel={ROLE_LEVEL.PROJECT_MANAGER}><ManagementDashboardPage /></RequireRoleLevel>`

כאן `children` הוא המסך המוגן.

> **שימו לב: הגנה בצד הלקוח אינה אבטחה**
>
> הבדיקות האלה משפרות חוויית משתמש בלבד (מסתירות מסכים). מי שמנסה לעקוף אותן עדיין נעצר בשרת, שבודק הרשאות בכל בקשה. תמיד אבטחה אמיתית נמצאת ב-Backend.

טעינה עצלה: lazy ו-Suspense

ב-`App.jsx` כל מסך נטען עם `lazy(() => import(...))`. הדפדפן מוריד את הקוד של מסך רק כשהמשתמש נכנס אליו — והאפליקציה נפתחת מהר יותר. בזמן ההורדה, `<Suspense fallback={...}>` מציג ספינר.

```
const MyProjectsPage = lazyNamed(() => import('./features/projects/MyProjectsPage'), 'MyProjectsPage');

<Suspense fallback={<CircularProgress aria-label="Loading page" />}>
  <Routes> ... </Routes>
</Suspense>
```

פרמטרים ב-query string

```
import { useSearchParams } from 'react-router-dom';

const [params, setParams] = useSearchParams();      // /tasks?status=OPEN&page=2
const status = params.get('status');                  // 'OPEN'
setParams({ status: 'DONE' });                        // navigates to /tasks?status=DONE
```

נסו בעצמכם 13.1

פתחו את `App.jsx` ותענו: (א) איזו קומפוננטה מוצגת בכתובת `/tasks/t9`? (ב) מה קורה למשתמש לא מחובר שנכנס ל-`/projects`? (ג) איזה תפקיד מינימלי נדרש ל-`/management/dashboard`? (ד) איפה מוגדר ה-`Outlet` שבו מוצג המסך?

נתונים מהשרת עם TanStack Query

רוב המסכים בפרויקט מציגים נתונים מהשרת. הספרייה הזו מנהלת טעינה, מטמון, ריענון ושגיאות — כדי שלא נצטרך `useEffect` ו-`useState` לכל בקשה.

מה הבעיה שהיא פותרת

בלי ספרייה, כל בקשה דורשת: state ל-data, state ל-loading, state לשגיאה, effect, cleanup, טיפול ב-race, ושמירה כדי לא להביא שוב את אותו דבר. TanStack Query (לשעבר React Query) עושה את כל זה. היא מבחינה בין **state של הממשק** (סרגל פתוח? טקסט בשדה?) שנשאר ב-`useState`, לבין **state של השרת** (רשימת הפרויקטים) שהיא מנהלת.

הגדרה כללית

בפרויקט שלנו: src/app/queryClient.js

```
export const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 15_000,            // data is "fresh" for 15 seconds - no refetch in that time
      refetchOnWindowFocus: false,  // don't refetch when the user returns to the tab
      retry: 1,                     // retry a failed request once
    },
  },
});
```

ה-`QueryClient` הוא המטמון המרכזי, ומוזרם לכל האפליקציה דרך `QueryClientProvider` ב-`main.jsx` (Context, כמו בפרק 12). `15_000` = 15000, עם קו תחתון להפרדה.

useQuery: קריאת נתונים

```
const { data, isLoading, isError, error, isFetching, refetch } = useQuery({
  queryKey: ['project', projectId],          // the cache key
  queryFn: () => fetchProject(projectId),    // returns a Promise with the data
  enabled: !!projectId,                      // only run when we have an id
});
```

שדה	משמעות
data	התוצאה. `undefined` עד שהגיעה — **תמיד** טפלו במצב הזה.
isLoading	true בזמן הטעינה הראשונה (אין עדיין נתונים).
isFetching	true בכל בקשה, גם ריענון ברקע כשכבר יש נתונים ישנים.
isError / error	הבקשה נכשלה (אחרי ניסיונות חוזרים).

queryKey: הלב של המערכת

ה-`queryKey` הוא מערך שמזהה את הנתון במטמון. שתי כתיבות זהות = אותה שאילתה = אותם נתונים משותפים. **כל ערך שהבקשה תלויה בו חייב להיות במפתח**; כשאחד מהם משתנה, זה מפתח חדש והספרייה מביאה נתונים חדשים.

```
['projects']                                  // all projects
['project', 'p7']                             // a single project
['projects', 'org1', { status: 'ACTIVE' }]    // filtered list
['projects', 'org1', { status: 'AT_RISK' }]   // different key -> separate cache entry + request
```

בפרויקט שלנו: src/lib/queries.js

```
export function useProjects(filters = {}) {
  const { organization } = useAuth();
  return useQuery({
    queryKey: ['projects', organization?.id, filters],
    queryFn: () => realApi.fetchProjects({ organizationId: organization.id, ...filters }),
    enabled: !!organization,           // wait until the organization is known
  });
}

export function useProject(projectId) {
  return useQuery({
    queryKey: ['project', projectId],
    queryFn: () => realApi.fetchProject(projectId),
    enabled: !!projectId,
  });
}
```

- המפתח של `useProjects` כולל את `filters`. כש-`MyProjectsPage` מעדכן את החיפוש, האובייקט `filters` משתנה ← מפתח חדש ← בקשה חדשה לשרת עם הסינון. אין כאן `useEffect`!
- `enabled: !!organization` — אל תפעיל עד שהארגון נטען (אחרת `organization.id` יזרוק שגיאה).
- `realApi.fetchProjects` נמצא ב-`src/api/api.js` ומשתמש ב-`http` (axios) שראינו בפרק 6.

שימוש במסך: שלושת המצבים

```
function ProjectsPage() {
  const { data: projects, isLoading, isError } = useProjects();

  if (isLoading) return <LoadingState label="Loading projects…" />;
  if (isError)   return <ErrorState onRetry={refetch} />;
  if (projects.length === 0) return <EmptyState title="No projects yet" />;

  return projects.map((p) => <ProjectCard key={p.id} project={p} />);
}
```

המבנה "empty → error → loading → תוכן" חוזר בכל מסך בפרויקט. הרכיבים `LoadingState` ו-`EmptyState` נמצאים ב-`components/common/States.jsx`.

שימו לב לביטוי `const { data: projects } = ...` — destructuring עם **שינוי שם** (פרק 4): "קח את `data` והצב אותו במשתנה `projects`". ולעיתים `const { data: filtered = [] }` — עם ערך ברירת מחדל בזמן שאין נתונים (כמו ב-`MyProjectsPage`).

useMutation: שינוי נתונים בשרת

לשאילתות קוראים; ל**פעולות** (יצירה, עדכון, מחיקה) משתמשים ב-`useMutation`. הוא לא רץ אוטומטית — מפעילים אותו עם `mutate`.

```
const queryClient = useQueryClient();

const updateTask = useMutation({
  mutationFn: ({ taskId, changes }) => api.updateTask(taskId, changes),   // the request
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['tasks'] });   // "this data is stale - refetch it"
  },
});

// In the component:
<Button onClick={() => updateTask.mutate({ taskId: 't1', changes: { title: 'New' } })}
        disabled={updateTask.isPending}>
  Save
</Button>
```

הקסם הוא ב-`invalidateQueries`: אחרי שינוי מוצלח מסמנים את הנתונים הרלוונטיים כ"לא עדכניים", והספרייה מביאה אותם שוב אוטומטית — וכל מסך שמציג אותם מתעדכן. אין צורך לעדכן ידנית.

בפרויקט שלנו: queries.js

```
export function useUpdateAutomationRule() {
  const qc = useQueryClient();
  return useMutation({
    mutationFn: ({ ruleId, rule }) => realApi.updateAutomationRule(ruleId, rule),
    onSuccess: () => qc.invalidateQueries({ queryKey: ['automation-rules'] }),
  });
}
```

עדכון המטמון ישירות: setQueryData

לפעמים אין צורך לבקש מהשרת — כבר יש לנו את הנתון. `setQueryData` כותב אותו למטמון, וכל מי שמשתמש בו מתרנדר מחדש. זה בדיוק מה שקורה בהתראות בזמן אמת (פרק 11): כשמגיעה הודעה מה-WebSocket, מוסיפים אותה לרשימה במטמון:

```
queryClient.setQueryData(['notifications', user.id], (prev) => (prev ? [notification, ...prev] :
[notification]));
```

איך "לקרוא" מסך שמשתמש ב-TanStack Query

1. מצאו את הקריאות `use...()` בראש הקומפוננטה — אלה **הנתונים** שהמסך צריך.
2. הכנסו ל-`queries.js` וראו לכל אחד את ה-`queryFn`, ומשם ל-`api.js` לנתיב ה-HTTP.
3. חפשו `.mutate` — אלה **הפעולות** שהמסך מבצע.

בשלושה צעדים ידעתם מאילו endpoints המסך תלוי — בלי לקרוא שורת JSX אחת.

נסו בעצמכם 14.1

פתחו את לשונית Network בדפדפן, היכנסו ל-`/projects` והקלידו בשדה החיפוש. אילו בקשות נשלחות? איזה query string? הסבירו מדוע (רמז: `queryKey` ו-`filters`).

ממשק עם MUI

MUI (Material UI) היא ספריית רכיבים מוכנים: כפתורים, שדות, כרטיסים, טבלאות, דיאלוגים. רוב ה-JSX בפרויקט הוא רכיבי MUI.

איך משתמשים ברכיב

```
import Button from '@mui/material/Button';
import TextField from '@mui/material/TextField';
import AddIcon from '@mui/icons-material/Add';

<Button variant="contained" color="primary" startIcon={<AddIcon />} onClick={save}>
  Add Task
</Button>

<TextField label="Title" value={title} onChange={(e) => setTitle(e.target.value)}
           size="small" fullWidth required error={!!error} helperText={error} />
```

הכלל: **כל ההתנהגות והמראה מגיעים דרך props**. כדי להבין רכיב — פותחים את התיעוד ב-`mui.com` ורואים את רשימת ה-props. הערכים הנפוצים: `variant` (סגנון): `contained`, `outlined`, `text`), `color`, `size`, `disabled`, `fullWidth`.

רכיבי פריסה: Box, Stack, Grid

רכיב	מה זה	מתי
Box	`div` עם תמיכה ב-`sx`	מיכל כללי, "תיבה" לעיצוב
Stack	ערימה ב-flexbox עם רווחים	שורה או עמודה של אלמנטים
Grid	רשת רספונסיבית (12 עמודות)	כרטיסים בשורות, פריסת עמוד
Typography	טקסט עם סגנון מהערכה	כותרות ופסקאות
Card, Paper, Chip	כרטיס, משטח, תגית	תצוגה

```
// Stack: a row of items with 12px gaps, vertically centered
<Stack direction="row" spacing={1.5} alignItems="center">
  <Avatar />
  <Typography variant="h5">Atlas Portal</Typography>
  <Chip label="Active" size="small" />
</Stack>

// Responsive Stack: column on phones, row from "sm" (600px) up
<Stack direction={{ xs: 'column', sm: 'row' }} spacing={2}>...</Stack>

// Grid: 12-column layout
<Grid container spacing={2}>
  <Grid size={{ xs: 12, sm: 6, lg: 4 }}>...</Grid>   {/* full / half / third width */}
  <Grid size={{ xs: 12, sm: 6, lg: 4 }}>...</Grid>
</Grid>
```

בפרויקט שלנו

`MyProjectsPage` ב-`<Grid size={{ xs: 12, sm: 6, md: 3 }}>`: בטלפון (`xs`) הכרטיס תופס 12 מתוך 12 עמודות (כל השורה); מ-600px ומעלה — 6 (חצי); מ-900px (רבע), כלומר ארבעה כרטיסי KPI בשורה. שימו לב: בגרסה 7 של MUI משתמשים ב-`size` (ולא בשיטה הישנה `xs=...` ישירות על `Grid item`).

נקודות עצירה (breakpoints): `xs`=0, `sm`=600, `md`=900, `lg`=1200, `xl`=1536. ערך שנכתב כאובייקט `{ xs: 1, md: 3 }` הוא "1 בקטן, 3 מ-md ומעלה".

ה-prop `sx`: עיצוב בתוך הרכיב

ל-`sx` מעבירים אובייקט עם תכונות CSS (ב-camelCase) ועוד קיצורים של MUI. זה הדבר שנראה הכי "צפוף" בקוד, אבל קל לפענח אם מכירים את הכללים:

כתיבה ב-sx	משמעות
mt: 2, mb: 1, px: 3, p: 0.5	margin-top / margin-bottom / padding אופקי / padding. **כל יחידה = 8px** (כך ש-`mt: 2` = 16px).
display: 'flex'	CSS רגיל (camelCase במקום מקף).
color: 'text.secondary'	צבע מה-theme: `text.secondary`, `error.main`, `success.main`, `primary.main`.
fontWeight: 700	משקל גופן.
'& svg': { fontSize: 18 }	"כל `svg` בתוך האלמנט הזה" (הסימן `&` = האלמנט עצמו).
'&:hover': { ... }	עיצוב ב-hover.
width: { xs: '100%', md: 400 }	ערך רספונסיבי.

בפרויקט שלנו: לפענח את KpiCard

KpiCard.jsx

```
<Typography variant="caption" color="text.secondary" fontWeight={600}
            sx={{ textTransform: 'uppercase', letterSpacing: 0.5 }}>
  {label}
</Typography>

<Box sx={{ color: accentColor, opacity: 0.85, display: 'flex', '& svg': { fontSize: 18 } }} aria-
hidden>
  {icon}
</Box>

<Typography variant="h2" sx={{ mt: 1, fontWeight: 700 }}>{value}</Typography>
```

קראו: "כותרת קטנה, אפורה, באותיות גדולות עם ריווח; אייקון בצבע המבטא שהוא flex וכל ה-svg בתוכו בגודל 18; מספר גדול, מודגש, עם מרווח עליון 8 פיקסלים". ה-`aria-hidden` מסתיר את האייקון מקוראי מסך (נגישות).

Theme: עיצוב מרכזי

הקובץ `src/theme/theme.js` מגדיר צבעי מותג, גופנים ובריירות מחדל לרכיבים (`createTheme`). ה-`ThemeProvider` ב-`main.jsx` מפיץ אותו, ולכן `color="primary"` או `'primary.main'` נותנים את הכחול של FlowBoard בכל מקום, ושינוי אחד בקובץ ה-theme משנה את כל האתר.

עוד רכיבים שתפגשו

- **Dialog** — חלון מודאלי: `<Dialog open={open} onClose={close}>`. ב-`ProjectFormDialog.jsx` תראו טופס בתוך דיאלוג, שנפתח ונסגר עם `useState(false)` בהורה (`createOpen` / `setCreateOpen` בלוח ה-Kanban).
- **Table** — טבלאות נתונים, למשל במסכי הניהול והדוחות.
- **Menu, Tooltip, Snackbar, Alert** — תפריטים, רמזים והודעות.
- **@mui/icons-material** — אייקונים: `import AddIcon from '@mui/icons-material/Add'`.
- **slotProps** — התאמת חלק פנימי של רכיב, כמו `slotProps={{ input: { startAdornment: ... } }}` ב-`TextField` להוספת אייקון חיפוש.

איך קוראים רכיב MUI שלא מכירים

חפשו בגוגל "MUI <שם הרכיב>" ופתחו את דף ה-API. שם יש טבלה של כל ה-props וה-`sx` שאפשר להעביר. בפועל תפגשו כ-15 רכיבים שחוזרים כל הזמן — ותכירו אותם תוך יום.

נסו בעצמכם 15.1

שנו ב-`KpiCard.jsx` את גודל המספר (`variant="h2"`) ל-`h1` וצבעו אותו באדום, ותראו את השינוי מיד בדפדפן. אחר כך הוסיפו ל-`Card` אפקט: `sx={{ '&:hover': { boxShadow: 6 } }}`.

תבניות מתקדמות שתפגשו

עוד כמה מושגים שמופיעים בפרויקט ובפרויקטים אחרים ושווה להיות מוכנים אליהם.

ErrorBoundary: תפיסת שגיאות ברינדור

אם קומפוננטה זורקת שגיאה בזמן הרינדור, בלי הגנה כל האפליקציה נעלמת (מסך לבן). **Error Boundary** תופס את השגיאה ומציג מסך חלופי. זה המקום היחיד שבו עדיין נדרשת **class component** — מבנה ישן יותר של React שבו הקומפוננטה היא מחלקה עם שיטת `render()`.

src/app/ErrorBoundary.jsx (simplified)

```
export class ErrorBoundary extends Component {
  state = { hasError: false };

  static getDerivedStateFromError() {       // called when a child throws
    return { hasError: true };
  }

  componentDidCatch(error, info) {          // a place to log the error
    console.error('FlowBoard prototype crashed:', error, info);
  }

  render() {
    if (this.state.hasError) {
      return <Box>Something went wrong <Button>Reload</Button></Box>;
    }
    return this.props.children;             // normal case: just render the children
  }
}
```

מקריאה: `this.state` ו-`this.props` הם ה-state וה-props של המחלקה. מספיק שתזהו את התבנית ("עוטף ומציג גיבוי כשמשהו קורס"). ב-`main.jsx` הוא עוטף את כל האפליקציה.

Drag and Drop עם dnd-kit

לוח ה-Kanban משתמש ב-`@dnd-kit/core`. שלושה רעיונות: `DndContext` עוטף את כל האזור הנגרר, `useDraggable` / `useSortable` הופך אלמנט לגריר, ו-`useDroppable` הופך אזור ליעד. כשמשחררים — נקרא `onDragEnd` עם `active` (הנגרר) ו-`over` (היעד).

```
// KanbanBoardPage.jsx
const sensors = useSensors(
  useSensor(PointerSensor, { activationConstraint: { distance: 6 } }),   // start dragging after 6px
  useSensor(KeyboardSensor),                                             // keyboard accessibility
);

const handleDragEnd = (event) => {
  const { active, over } = event;                        // active = the card, over = the column
  const move = resolveTaskMove(tasks ?? [], String(active.id), over ? String(over.id) : null);
  if (move) {
    moveTask.mutate(move, { onError: (err) => setMoveError(err.message) });
  }
};
```

```
<DndContext sensors={sensors} onDragEnd={handleDragEnd}>
  {columns.map((column) => <KanbanColumn key={column.id} column={column} ... />)}
</DndContext>

// KanbanColumn.jsx
const { setNodeRef, isOver } = useDroppable({ id: column.id });
<Box ref={setNodeRef} sx={{ border: isOver ? '2px dashed #3454D1' : '2px dashed transparent' }}>
```

שימו לב לזרימה: גרירה ← `onDragEnd` ← `resolveTaskMove` (פונקציה טהורה שמחליטה אם צריך בכלל לזוז) ← `moveTask.mutate` (בקשה לשרת) ← `invalidateQueries` ← הלוח מתעדכן. הפרדת ההחלטה לפונקציה נפרדת מקלה על בדיקה: ראו `resolveTaskMove.test.js`.

פונקציות טהורות ובדיקות

פונקציה **טהורה** תלויה רק בארגומנטים ולא משנה דבר חיצוני. קל לבדוק אותן, ולכן מומלץ להוציא לוגיקה מהקומפוננטות אליהן (`resolveTaskMove`, `format.js`, `permissions.js`). בפרויקט הבדיקות רצות עם **Vitest** (`npm test`) והן נראות כך (דוגמה מפושטת בסגנון הבדיקות בפרויקט):

```
import { describe, it, expect } from 'vitest';
import { resolveTaskMove } from './resolveTaskMove';

describe('resolveTaskMove', () => {
  it('returns null when the task is dropped on its own column', () => {
    const tasks = [{ id: 't1', columnId: 'c1' }];
    expect(resolveTaskMove(tasks, 't1', 'c1')).toBeNull();
  });
  it('returns the move payload for a different column', () => {
    const tasks = [{ id: 't1', columnId: 'c1' }];
    expect(resolveTaskMove(tasks, 't1', 'c2')).toEqual({ taskId: 't1', columnId: 'c2' });
  });
});
```

הקריאה: `describe` = קבוצה, `it` = בדיקה אחת בשפה טבעית, `expect(...).toEqual(...)` = טענה. בדיקה שנכשלת אומרת לכם בדיוק איזו התנהגות נשברה.

גרפים עם Recharts

ב-`KpiCard` יש גרף קטן. Recharts בנויה מרכיבים מקוננים: מיכל, סוג גרף, ורכיבים לציר וסדרות. הנתונים הם **מערך אובייקטים**, וכל סדרה מצביעה על שדה בשם.

```
<ResponsiveContainer width="100%" height="100%">          {/* fills its parent's size */}
  <AreaChart data={[{ v: 3 }, { v: 5 }, { v: 4 }, { v: 8 }]}>
    <Area type="monotone" dataKey="v" stroke="#3454D1" fill="url(#grad)" />   {/* dataKey = which field
*/}
  </AreaChart>
</ResponsiveContainer>
```

שימו לב ל-`sparkline.map((v) => ({ v }))` ב-`KpiCard`: ממירים מערך מספרים `[3,5,4]` למערך אובייקטים `[{v:3}, ...]` כי Recharts דורשת אובייקטים. וב-`AreaChart` בפועל הסוגריים `({...})` הם "החזרת אובייקט מ-arrow" (פרק 3).

דפוסים קטנים נוספים

```
// Spread props: pass everything else through
function IconButton({ label, ...rest }) {
  return <button aria-label={label} {...rest} />;      // onClick, disabled, etc. flow through
}

// A component receiving another component as a prop
<EmptyState icon={<FolderOffOutlinedIcon />} title="No projects" action={<Button>Create</Button>} />

// Render props / function as a child (older pattern)
<DataLoader url="/x">{(data) => <List items={data} />}</DataLoader>

// Passing "wait for it" defaults
const { data: filtered = [] } = useProjects(...);       // [] until loaded, so .map never crashes

// Boolean filter idiom
users.map((id) => userMap.get(id)).filter(Boolean)      // drops undefined / null

// Changing key resets a component's state
<Editor key={taskId} task={task} />                     // new taskId = a brand-new Editor
```

נגישות (Accessibility) בקוד

בפרויקט תראו מאפיינים כמו `aria-label`, `role="status"`, `aria-hidden` ו-`alt`. אלה מידע לקוראי מסך וטכנולוגיות מסייעות. הפרויקט אף בודק נגישות אוטומטית עם `axe-core`. כשאתם רואים אותם — הבינו שהם לא משפיעים על המראה, רק על מי ש"קורא" את הדף.

נסו בעצמכם 16.1

קראו את `src/features/kanban/KanbanTaskCard.jsx` ופרטו: אילו hooks של dnd-kit הוא משתמש בהם? מה קורה כש-`canDragTask(task)` הוא false? אילו נתונים הוא מקבל ב-props ומה כל אחד מהם משמש?

סיור מודרך: קריאת דף שלם

נקרא יחד קובץ אמיתי מהפרויקט, `src/features/projects/MyProjectsPage.jsx`, מההתחלה ועד הסוף — ונעקוב אחרי מה שקורה כשמשתמש מקליד בשדה החיפוש.

שלב 1: הקובץ מלמעלה — ה-imports

MyProjectsPage.jsx, lines 1-26 (abridged)

```
import { useMemo, useState } from 'react';
import Box from '@mui/material/Box';
import Grid from '@mui/material/Grid';
import TextField from '@mui/material/TextField';
import Stack from '@mui/material/Stack';
import Chip from '@mui/material/Chip';
import Button from '@mui/material/Button';
import SearchIcon from '@mui/icons-material/Search';
...
import { useNavigate } from 'react-router-dom';
import { PageHeader } from '../../components/common/PageHeader';
import { ProjectCard } from '../../components/common/ProjectCard';
import { KpiCard } from '../../components/common/KpiCard';
import { EmptyState, LoadingState, NoSearchResultsState } from '../../components/common/States';
import { useAuth } from '../../auth/AuthContext';
import { useOrganizationUsers, useProjects } from '../../lib/queries';
import { isManager } from '../../lib/permissions';
import { buildSparkline } from '../../lib/trends';
```

מה למדנו לפני שקראנו שורה אחת של לוגיקה:

- `useMemo, useState` — יש state וחישובים מבוססי-תלויות.
- רכיבי MUI (`Box`, `Grid`, `TextField`, `Chip`, `Stack`) — יש פריסה, שדה קלט ותגיות.
- `useNavigate` — הדף מנווט לדפים אחרים.
- רכיבים משלנו: `PageHeader`, `ProjectCard`, `KpiCard`, מצבי טעינה/ריק.
- `useAuth`, `useProjects`, `useOrganizationUsers` — נתונים מהשרת ופרטי המשתמש המחובר.
- `isManager` — תלוי-הרשאות.

שלב 2: קבועים מחוץ לקומפוננטה

```
const STATUS_FILTERS = [
  { label: 'All', value: 'ALL' },
  { label: 'Active', value: 'ACTIVE' },
  { label: 'At Risk', value: 'AT_RISK' },
  ...
];
```

נתונים שלא משתנים מוגדרים **מחוץ** לפונקציה, כדי שלא ייווצרו מחדש בכל רינדור. כל `label` הוא הטקסט שהמשתמש רואה, וה-`value` הוא הערך שנשלח לשרת. בהמשך `.map` יהפוך את המערך לצ׳יפים.

שלב 3: תחילת הקומפוננטה — הוקים וגזירת נתונים

```
export function MyProjectsPage() {
  const { user, activeRole } = useAuth();          // (a) who is signed in, and in which role
  const navigate = useNavigate();                  // (b) a function that changes the URL
  const [search, setSearch] = useState('');        // (c) UI state: the search box text
  const [status, setStatus] = useState('ALL');     // (d) UI state: the selected chip

  const { data: projects, isLoading } = useProjects({ memberId: user?.id });     // (e)
  const { data: filtered = [] } = useProjects({                                   // (f)
    memberId: user?.id,
    status: status === 'ALL' ? undefined : status,
    search: search.trim() || undefined,
  });
  const { data: users } = useOrganizationUsers();                                 // (g)
  const userMap = useMemo(() => new Map((users ?? []).map((u) => [u.id, u])), [users]);   // (h)

  const summary = useMemo(() => {                                                 // (i)
    const list = projects ?? [];
    return {
      total: list.length,
      active: list.filter((p) => p.status === 'ACTIVE').length,
      ...
    };
  }, [projects]);

  if (isLoading) return <LoadingState label="Loading your projects…" />;          // (j)
  ...
```

#	מה קורה	מושג מהמדריך
a	מקבלים את המשתמש והתפקיד הפעיל מה-Context.	פרק 12
b	`navigate` משמש בהמשך בלחיצת כפתור.	פרק 13
c, d	שני `useState` — ערך השדה והסטטוס הנבחר. הם מה שהמשתמש שולט בו.	פרק 10
e	שליפה של **כל** הפרויקטים של המשתמש (ללא סינון) — לכרטיסי הסיכום ולבדיקה "אין לי פרויקטים".	פרק 14
f	שליפה שנייה, **מסוננת בצד השרת** לפי הסטטוס והחיפוש. `undefined` = "אל תשלח את הפרמטר". `= []` = ברירת מחדל בזמן שאין נתונים.	פרקים 2, 14
g	רשימת משתמשי הארגון, כדי להציג שמות ותמונות.	פרק 14
h	ממירים מערך משתמשים למילון לפי id, ומחשבים מחדש רק כש-`users` משתנה.	פרקים 4, 11
i	מחשבים ספירות מהרשימה עם `filter().length`.	פרק 4
j	יציאה מוקדמת: כל עוד הטעינה הראשונה לא הסתיימה — מציגים מסך טעינה.	פרק 9

שימו לב: הוקים תמיד לפני ה-`return` המוקדם

בפרק 11 ראינו שאסור לקרוא להוקים בתוך תנאי. לכן כל ה-`use...` מופיעים **לפני** ה-`if (isLoading) return ...`, ולא אחריו. אם תעבירו הוק אחרי ה-return המוקדם — תקבלו שגיאת "Rendered more hooks than during the previous render".

שלב 4: ה-JSX — קוראים מהשורש פנימה

```
return (
  <Box>
    <PageHeader
      title="My Projects"
      description="Projects you are a member of."
      actions={
        isManager(activeRole) ? (
          <Button variant="outlined" onClick={() => navigate('/management/projects')}>
            Manage all projects
          </Button>
        ) : undefined
      }
    />

    {(projects ?? []).length > 0 && (
      <Grid container spacing={2} sx={{ mb: 2.5 }}>
        <Grid size={{ xs: 12, sm: 6, md: 3 }}>
          <KpiCard label="Total Projects" value={summary.total}
                   icon={<FolderOutlinedIcon fontSize="small" />} sparkline=
{buildSparkline(summary.total, 6, 1)} />
        </Grid>
        ...three more KpiCards...
      </Grid>
    )}
```

- **PageHeader** מקבל `title`, `description` ו-`actions` — ו-`actions` הוא **אלמנט JSX** שמועבר כ-prop. הטרנרי אומר: מנהלים רואים כפתור "Manage all projects", כל השאר `undefined` (לא מוצג כלום).
- הבלוק `{(projects ?? []).length > 0 && (...)}` — כרטיסי הסיכום מוצגים רק אם יש לפחות פרויקט אחד. שימו לב לטיפול הנכון ב-`&&`: משווים ל-`> 0` ולא מסתמכים על המספר עצמו.
- כל `KpiCard` מקבל את ה-props שראינו בחתימה (פרק 3): `label`, `value`, `icon` (אלמנט), `sparkline` (מערך).

```
    <Stack direction={{ xs: 'column', sm: 'row' }} spacing={1.5} sx={{ mb: 2.5 }}>
      <TextField
        size="small"
        placeholder="Search projects…"
        value={search}                                    // controlled input (chapter 10)
        onChange={(e) => setSearch(e.target.value)}
      />
      <Stack direction="row" spacing={1} flexWrap="wrap" useFlexGap>
        {STATUS_FILTERS.map((f) => (                      // one Chip per filter (chapter 9)
          <Chip
            key={f.value}
            label={f.label}
            onClick={() => setStatus(f.value)}
            color={status === f.value ? 'primary' : 'default'}       // highlight the selected one
            variant={status === f.value ? 'filled' : 'outlined'}
          />
        ))}
      </Stack>
    </Stack>

    {(projects ?? []).length === 0 ? (
      <EmptyState title="You're not part of any project yet" ... />
    ) : filtered.length === 0 ? (
      <NoSearchResultsState onClear={() => { setSearch(''); setStatus('ALL'); }} />
    ) : (
```

```
      <Grid container spacing={2}>
        {filtered.map((p) => (
          <Grid key={p.id} size={{ xs: 12, sm: 6, lg: 4 }}>
            <ProjectCard project={p} manager={userMap.get(p.managerId)}
                         members={p.memberIds.map((id) => userMap.get(id)).filter(Boolean)} />
          </Grid>
        ))}
      </Grid>
    )}
  </Box>
);
```

- שדה החיפוש הוא **controlled**: `value={search}` + `onChange`.
- הצ׳יפים נוצרים מ-`STATUS_FILTERS.map`. הצ׳יפ הנבחר מקבל `color="primary"`; לחיצה קוראת `setStatus`.
- שלושת המצבים של התוכן — אין פרויקטים / אין תוצאות לסינון / יש תוצאות — בשרשרת טרנרי אחת.
- הכפתור **"clear"** ב-`NoSearchResultsState` מקבל פונקציה שמאפסת את שני ה-state.

מעקב: מה קורה כשמקלידים "atl" בשדה החיפוש

1. המשתמש מקליד "a" ◀ מופעל `onChange` עם `e.target.value = "a"`
2. `setSearch("a")` ◀ React מתזמנת רינדור חדש
3. `MyProjectsPage()` רצה שוב, ועכשיו `search === "a"`
4. נקראת `useProjects({ memberId, status: undefined, search: "a" })`
5. נוצר `queryKey` חדש ◀ `['projects', orgId, { ..., search: 'a' }]`
6. TanStack Query בודקת את המטמון — אין ◀ שולחת `GET /api/projects?search=a`
7. עד שהתשובה מגיעה, `filtered` נשאר כמו שהיה
8. התשובה מגיעה ◀ `filtered` מתעדכן ◀ הרשת מציגה את התוצאות
9. אותו תהליך חוזר עבור "at" ועבור "atl"

לא כתבנו אף `useEffect` ואף `fetch`. הכול נגזר מ-`state` ומה-`queryKey`. זו הסיבה שקוד React מודרני נראה "קצר" — הנתונים זורמים מ-state, דרך הוקים, אל ה-JSX.

בדקו בעצמכם בדפדפן

1. הריצו את הפרויקט (`npm run dev`) והתחברו.
2. פתחו `F12` ← Network ← סננו "projects".
3. הקלידו בשדה החיפוש ותראו בקשה חדשה בכל שינוי, עם `?search=...`.
4. לחצו על צ׳יפ "Active" ותראו `?status=ACTIVE`.

איך משנים משהו: שלושה תרגילי שינוי

17.1 — לשנות

1. הוסיפו צ׳יפ סטטוס חדש בשם "Archived" עם ערך `'ARCHIVED'` (שורה אחת ב-`STATUS_FILTERS`).
2. שנו את הכותרת "My Projects" ל"הפרויקטים שלי". איפה חיפשתם אותה? (רמז: Ctrl+Shift+F).
3. הוסיפו מתחת לשדה החיפוש טקסט קטן: `Showing {filtered.length} of {summary.total} projects`.

איך קוראים קוד זר

היכולת שלשמה כתבנו את המדריך: לפתוח פרויקט שמעולם לא ראיתם ולהבין אותו. זו מיומנות שאפשר ללמוד — והנה השיטה.

שיטה בעשרה צעדים

1. **הריצו אותו.** ראו את האפליקציה בפעולה לפני שאתם קוראים קוד. תבינו מה כל מסך אמור לעשות.
2. **פתחו את** `package.json` — אילו ספריות? (React? Router? MUI? Redux? Tailwind) זה קובע מה תקראו.
3. **מצאו את נקודת הכניסה** (`main.jsx` / `index.js`) וראו אילו Providers עוטפים את האפליקציה.
4. **מצאו את טבלת הנתיבים** (`App.jsx` / `routes`) — מפה של כל המסכים והקומפוננטות שלהם.
5. **בחרו מסך אחד** — זה שמעניין אתכם — ופתחו את הקומפוננטה שלו.
6. **קראו imports ראשונים** — הם מסכמים את התלויות (פרק 5).
7. **קראו את החתימה** של הקומפוננטה: אילו props היא מקבלת?
8. **קראו את ההוקים בראש** (`useState`, `use...` מותאמים): מה ה-state ומה הנתונים?
9. **קראו את ה-JSX מלמעלה למטה** והתחילו מהמבנה, לא מהפרטים. אחר כך צללו לקומפוננטות ילדות לפי הצורך.
10. **שנו משהו קטן** ובדקו מה קורה. שינוי טקסט, צבע או תנאי הוא הדרך הכי מהירה לאמת שהבנתם.

שאלות שכדאי לשאול על כל קומפוננטה

שאלה	איפה מוצאים תשובה
איזה מידע היא מקבלת מבחוץ?	החתימה — ה-props
איזה מידע היא מנהלת בעצמה?	`useState` / `useReducer`
מאיפה הנתונים מהשרת?	הוקים כמו `useQuery` ו-`use...` ב-`queries.js`
מה קורה בלחיצה / שליחה?	`onClick`, `onSubmit` ← ה-handler ← `mutate` / `navigate`
מה מוצג בתנאים?	`&&`, `?:`, `if ... return`
מה קורה בעת טעינה / שגיאה / ריק?	ה-`if`ים לפני ה-JSX הראשי
למי היא יכולה להיות גלויה?	`isManager`, `RequireRoleLevel`, `useAuth`

קריאת שגיאות

שגיאות בקונסולה נראות מפחידות, אבל בדרך כלל הסיפור בשורה הראשונה ובשורה הראשונה של ה-stack ששייכת לקוד **שלכם** (לא ל-`node_modules`).

הודעה	משמעות ופתרון
Cannot read properties of undefined (reading 'map')	ניסיתם `x.map` כש-`x` הוא `undefined` — כנראה נתונים שעוד לא נטענו. שימו `?? []` או `if (isLoading) return`.
x is not a function	קראתם לדבר שאינו פונקציה: prop שלא הועבר, או import שגוי (named מול default).
Objects are not valid as a React child	ניסיתם לרנדר אובייקט ב-`{ }`. הציגו שדה מסוים (`{user.name}`).
Each child in a list should have a unique "key" prop	חסר `key` ב-`map` (פרק 9).
Rendered more hooks than during the previous render	הוק אחרי `return` מוקדם או בתוך תנאי (פרק 11).
Too many re-renders	קראתם ל-`setState` בגוף הקומפוננטה (או `onClick={fn()}`) וכך גרמתם ללולאה אינסופית.
Network Error / 401 / 403 / 500	בדקו בלשונית Network: כתובת נכונה? השרת רץ? האם המשתמש מחובר? מה גוף התשובה?

כלי דיבוג

- **console.log** — הכלי הבסיסי. הדפיסו את מה שאתם חושדים בו: `console.log({ projects, filters })`.
- **React Developer Tools** — תוסף לדפדפן שמוסיף לשוניות "Components" ו-"Profiler". אפשר לראות את עץ הקומפוננטות, את ה-props וה-state של כל אחת, ואפילו לערוך אותם בזמן אמת. **הכלי הכי שימושי ללמידה.**
- **Network** — כל בקשה, התגובה שלה והזמנים.
- **Breakpoints** — ב-Sources לוחצים על מספר שורה, והריצה נעצרת שם ואפשר לצפות בערכי המשתנים.
- **TanStack Query Devtools** — (אם מוגדר) מציג את כל ה-queries והמצב שלהם.

דברים שמבלבלים בהתחלה

מה שרואים	איך לקרוא
style={{ a: 1 }}	סוגריים חיצוניים = "בא ביטוי JS", פנימיים = "אובייקט".
{cond && <X />}	"הצג X רק אם cond".
onClick={() => f(id)}	"בלחיצה — הרץ f עם id".
const [a, setA] = useState()	"תא זיכרון a, ופונקציה לעדכנו".
useEffect(..., [x])	"הרץ אחרי רינדור, וכש-x משתנה".
a?.b ?? c	"b של a אם a קיים, ואחרת c".
{ ...obj, k: v }	"עותק של obj, עם k מעודכן".
export function X({ a, b })	"קומפוננטה X עם props a ו-b".
.map(...).filter(Boolean)	"הפוך כל איבר, ואז זרוק ריקים".

מה ללמוד הלאה

- **TypeScript** — JavaScript עם טיפוסים. רוב הפרויקטים המקצועיים כותבים בו. כל מה שלמדתם תקף; רק מוסיפים `: string` ו-`interface`.
- **ניהול טפסים** — `react-hook-form` עם `zod` לוולידציה.
- **Redux / Zustand** — ניהול state גלובלי (בפרויקט שלנו Context ו-TanStack Query מספיקים).

- **Next.js** — פריימוורק שבנוי על React, עם רינדור בצד השרת ונתיבים מבוססי תיקיות.
- **בדיקות** — Vitest / Jest עם React Testing Library לבדיקת קומפוננטות; Playwright לבדיקות מקצה לקצה (כפי שיש בפרויקט).
- **CSS** — Flexbox ו-Grid מעבר ל-`sx` של MUI; Tailwind כאלטרנטיבה.
- **Git** — לשמור גרסאות ולעבוד בצוות.

מקורות מומלצים

אתר	מה יש בו
react.dev	התיעוד הרשמי של React, עם מדריך אינטראקטיבי מצוין ("Learn").
developer.mozilla.org	MDN — ההפניה המובילה ל-HTML, JavaScript ו-CSS.
javascript.info	מדריך JavaScript מודרני מקיף.
tanstack.com/query	תיעוד TanStack Query.
reactrouter.com	תיעוד React Router.
mui.com	התיעוד של MUI, עם דוגמאות חיות לכל רכיב.

מילה אחרונה

כולם מתחילים בלקרוא קוד שהם לא מבינים. ההבדל בין מתחיל לבעל ניסיון הוא לא ידע — הוא הרגל לשאול "מאיפה הערך הזה מגיע ומה משנה אותו?". תעשו את זה בכל שורה, ותראו איך הקוד "נפתח".

נספחים

עזרים

דף עזר מהיר, מילון מונחים ופתרונות לתרגילים.

נספח א׳

דף עזר מהיר

מומלץ להדפיס ולהחזיק ליד המקלדת בזמן שקוראים קוד.

JavaScript

נושא	תחביר
משתנים	const a = 1; let b = 2;
Template literal	`Hi ${name}, ${n + 1}`
Arrow function	(a, b) => a + b | x => ({ id: x })
ברירת מחדל	function f(a = 0, { x, y = 1 } = {}) {}
Destructuring	const { a, b: c = 5 } = obj; const [x, y] = arr;
Spread / rest	{ ...obj, k: 1 } [...arr, 4] (...args) => {}
Optional chaining	a?.b?.[0]?.()
Nullish	a ?? 'default' (רק null/undefined)
Truthy AND / OR	a && b | a || b
טרנרי	cond ? x : y
מערך: המרה	arr.map(fn) arr.filter(fn) arr.reduce(fn, init)
מערך: חיפוש	arr.find(fn) arr.some(fn) arr.every(fn) arr.includes(x)
מערך: מיון (משנה!)	[...arr].sort((a, b) => a - b)
אובייקט	Object.keys / values / entries(obj)
JSON	JSON.parse(text) JSON.stringify(obj)
Export	export const a = 1; export default fn;
Import	import D, { a, b as c } from './x'; import * as X from './x';
Promise	p.then(f).catch(g).finally(h)
async / await	async function f() { try { await x; } catch (e) {} }
מקביל	await Promise.all([p1, p2])

React

נושא	תחביר
קומפוננטה	function Card({ title, children }) { return ... ; }
ביטוי ב-JSX	{value}
Class / style	className="a" style={{ color: 'red' }}
רשימה	{items.map(i => <Row key={i.id} {...i} />)}
תנאי	{ok ? <A /> : <B />} {ok && <A />}
State	const [v, setV] = useState(init); setV(p => p + 1)
אירוע	onClick={() => f(id)} onChange={e => set(e.target.value)}
Effect	useEffect(() => { ...; return cleanup; }, [deps]);
Memo	useMemo(() => compute(a), [a])
Ref	const r = useRef(null); <input ref={r} /> r.current
Context	createContext() <C.Provider value={v}> useContext(C)

Router, Query, MUI

נושא	תחביר
Route	<Route path="/p/:id" element={<Page />} />
פרמטרים	const { id } = useParams();
ניווט	const go = useNavigate(); go('/x'); go(-1);
קישור / הפניה	<Link to="/x" /> <Navigate to="/x" replace />
Outlet	<Outlet /> (בתוך layout route)
שאילתה	useQuery({ queryKey: ['k', id], queryFn, enabled })
פעולה	useMutation({ mutationFn, onSuccess }) m.mutate(vars)
ריענון	qc.invalidateQueries({ queryKey: ['k'] })
Stack / Grid	<Stack direction="row" spacing={2} /> <Grid size={{ xs: 12, md: 6 }} />
sx	sx={{ mt: 2, px: 3, color: 'text.secondary', '&:hover': {...} }}

יחידות

ב-`sx`: `1` = 8px. נקודות שבירה: `xs` 0 · `sm` 600 · `md` 900 · `lg` 1200 · `xl` 1536.

מילון מונחים

מונח	הסבר קצר
API	ממשק שבו ה-Frontend מבקש נתונים מהשרת (כתובות כמו `/api/projects`).
Arrow function	פונקציה בתחביר `(a) => ...`.
async / await	תחביר לכתיבת קוד אסינכרוני כאילו היה סינכרוני.
Backend / Frontend	צד שרת / צד לקוח (מה שרץ בדפדפן).
Bundler (Vite)	כלי שמאגד את הקבצים לקוד שהדפדפן טוען ומספק שרת פיתוח.
Callback	פונקציה שמועברת לפונקציה אחרת כדי שתיקרא בהמשך.
Closure	פונקציה ש"זוכרת" את המשתנים מהסביבה שבה נוצרה.
Component	פונקציה שמחזירה JSX; אבן הבניין של React.
Context	מנגנון React להעברת ערך לכל הרכיבים שמתחת ל-Provider בלי props.
Controlled input	שדה קלט שערכו נשלט על ידי state.
Destructuring	פירוק אובייקט או מערך למשתנים: `const {a} = obj`.
DOM	עץ האובייקטים שהדפדפן בונה מה-HTML.
Dependency array	המערך השני של `useEffect` / `useMemo`: מתי להריץ מחדש.
Effect	קוד שרץ אחרי רינדור, לפעולות "מחוץ" ל-React.
Hook	פונקציה `use...` שנותנת לקומפוננטה יכולות של React.
Immutability	לא לשנות נתונים קיימים אלא ליצור עותק מעודכן.
Interceptor	פונקציה ב-axios שרצה על כל בקשה/תשובה.
JSON	פורמט טקסט להעברת נתונים.
JSX	תחביר דמוי-HTML בתוך JavaScript.
Key	מזהה ייחודי לאיבר ברשימה ב-React.
Memoization	שמירת תוצאת חישוב כדי לא לחשב אותה שוב (`useMemo`).
Mutation	(1) שינוי ישיר באובייקט; (2) ב-TanStack Query: פעולה ששולחת שינוי לשרת.
Outlet	מקום ב-layout route שבו מוצג הנתיב הילד.
Promise	אובייקט שמייצג תוצאה עתידית של פעולה אסינכרונית.
Props	הנתונים שקומפוננטה מקבלת מההורה.
Provider	רכיב שמספק ערך של Context לילדיו.
Query key	מפתח מערך שמזהה נתון במטמון של TanStack Query.
Render	הרצת הקומפוננטה כדי לחשב איך המסך נראה.
SPA	אפליקציית דף יחיד, שמחליפה תוכן בלי לטעון דפים.
State	מידע שקומפוננטה זוכרת ושינויו גורם לרינדור מחדש.
StrictMode	מצב פיתוח של React שמריץ effects פעמיים כדי לחשוף באגים.
Theme	הגדרות עיצוב מרכזיות (צבעים, גופנים) של MUI.
Truthy / Falsy	ערכים שנחשבים אמת / שקר בתנאי.

פתרונות לתרגילים

נסו קודם בעצמכם. הפתרונות הם אחת מכמה אפשרויות נכונות.

2.1 — ביטויים

```
0 ?? 5          // 0        (0 is not null/undefined)
0 || 5          // 5        (0 is falsy)
'' ?? 'x'       // ''
null?.a         // undefined
[] && 'yes'     // 'yes'    (an empty array is truthy)
typeof NaN      // 'number'
```

3.1 — פונקציות

```
const isAdult = (age) => age >= 18;

const describe = ({ name, age = 18 }) => `${name} is ${age}`;

function makeCounter() {
  let count = 0;
  return () => ++count;      // closure over count
}
const next = makeCounter();
next(); next(); next();      // 1, 2, 3
```

4.1 — הזמנות

```
const unpaidTotal = orders.filter((o) => !o.paid).reduce((s, o) => s + o.total, 0);   // 195
const ids = orders.map((o) => o.id);                                                   // [1, 2, 3]
const hasBig = orders.some((o) => o.total > 100);                                      // true
const byId = Object.fromEntries(orders.map((o) => [o.id, o.total]));                   // {1:50,
2:120, 3:75}
```

6.1 — סדר ההדפסות

```
▸ פלט
A
D
F
C
E
B
```

קודם כל הקוד הסינכרוני (`A`, גוף הפונקציה `async` עד ה-`await` שמדפיס `D`, ואז `F`). אחר כך מיקרו-משימות לפי סדר הרישום: `then` (C) ואז ההמשך אחרי `await` (E). לבסוף `setTimeout` (B).

משימות — 8.1

```
const openTitles = tasks.filter((t) => !t.done).map((t) => t.title);
// ['Login page', 'Write docs', 'Refactor']

const perAssignee = tasks
  .filter((t) => t.assignee)
  .reduce((acc, t) => ({ ...acc, [t.assignee]: (acc[t.assignee] ?? 0) + 1 }), {});
// { dana: 2, omer: 1 }

const firstUrgent = tasks.find((t) => !t.done && ['URGENT', 'HIGH'].includes(t.priority));
// { id: 1, title: 'Login page', ... }

const danaAllOpen = tasks.filter((t) => t.assignee === 'dana').every((t) => !t.done);   // true
```

getUserName — 8.2

```
async function getUserName(id) {
  try {
    const res = await fetch(`https://jsonplaceholder.typicode.com/users/${id}`);
    if (!res.ok) return 'Unknown';
    const user = await res.json();
    return user.name;
  } catch {
    return 'Unknown';
  }
}
```

refreshPromise — 8.3

הפונקציה מונעת ריצה מקבילה של אותו רענון: אם כבר יש Promise פעיל, כל הקוראים האחרים מקבלים אותו במקום להתחיל רענון חדש. המשתנה מוגדר **מחוץ** לפונקציה כדי שישרוד בין קריאות (הוא "קופסה משותפת" — closure ברמת המודול), וב-`finally` מאפסים אותו כדי שרענון עתידי יוכל להתחיל.

UserList — 9.1

```
function UserList({ users }) {
  if (users.length === 0) return <p>No users</p>;
  return (
    <ul>
      {users.map((u) => (
        <li key={u.id} style={{ color: u.active ? 'inherit' : 'gray' }}>
          {u.name} {u.active && '✓'}
        </li>
      ))}
    </ul>
  );
}
```

TodoApp — 10.1

```
function TodoApp() {
  const [text, setText] = useState('');
  const [todos, setTodos] = useState([]);

  const add = () => {
    if (!text.trim()) return;
    setTodos([...todos, { id: Date.now(), text, done: false }]);
```

```
    setText('');
  };
  const toggle = (id) => setTodos(todos.map((t) => (t.id === id ? { ...t, done: !t.done } : t)));
  const remove = (id) => setTodos(todos.filter((t) => t.id !== id));
  const open = todos.filter((t) => !t.done).length;          // derived, not state

  return (
    <div>
      <input value={text} onChange={(e) => setText(e.target.value)} />
      <button onClick={add}>Add</button>
      <ul>
        {todos.map((t) => (
          <li key={t.id} style={{ textDecoration: t.done ? 'line-through' : 'none' }}>
            {t.text}
            <button onClick={() => toggle(t.id)}>✓</button>
            <button onClick={() => remove(t.id)}>✕</button>
          </li>
        ))}
      </ul>
      <p>{open} open</p>
    </div>
  );
}
```

useWindowWidth — 11.1

```
function useWindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);
  useEffect(() => {
    const onResize = () => setWidth(window.innerWidth);
    window.addEventListener('resize', onResize);
    return () => window.removeEventListener('resize', onResize);   // cleanup
  }, []);
  return width;
}

function Label() {
  const width = useWindowWidth();
  return <p>{width < 600 ? 'Mobile' : 'Desktop'}</p>;
}
```

CartContext — 12.1

```
const CartContext = createContext(undefined);

export function CartProvider({ children }) {
  const [items, setItems] = useState([]);
  const add = (item) => setItems((prev) => [...prev, item]);
  const remove = (id) => setItems((prev) => prev.filter((i) => i.id !== id));
  return <CartContext.Provider value={{ items, add, remove }}>{children}</CartContext.Provider>;
}

export function useCart() {
  const ctx = useContext(CartContext);
  if (!ctx) throw new Error('useCart must be used within CartProvider');
  return ctx;
}
```

```
function AddButton() { const { add } = useCart(); return <button onClick={() => add({ id: Date.now()
})}>Add</button>; }
function Counter()   { const { items } = useCart(); return <p>Items: {items.length}</p>; }
```

App.jsx — 13.1

- (א) `TaskDetailsPage` (הנתיב `/tasks/:taskId`).
- (ב) `ProtectedRoute` מפנה אותו ל-`/sign-in` עם `replace` ושומר את הכתובת המקורית ב-`state.from`.
- (ג) `ROLE_LEVEL.PROJECT_MANAGER` (רמה 2), דרך `RequireRoleLevel`.
- (ד) ב-`AppShell` (`src/layout/AppShell.jsx`).

Network — 14.1

כל הקלדה משנה את `search`, ולכן את ה-`filters` ואת ה-`queryKey`. TanStack Query רואה מפתח חדש ושולחת בקשה חדשה, למשל `GET /api/projects?search=at`. אם חוזרים לערך שכבר נשלח (וטרם פג ה-`staleTime` של 15 שניות) — התשובה נלקחת מהמטמון בלי בקשה.

15.1 ו-16.1 ו-17.1

אלה תרגילי חקירה ושינוי בקוד האמיתי — התוצאה נראית מיד בדפדפן. ב-17.1(א) מוסיפים `{ label: 'Archived', value: 'ARCHIVED' }` למערך `STATUS_FILTERS`; ב-17.1(ג) מוסיפים בתוך ה-JSX, מתחת ל-`Stack` של החיפוש:

`<Typography variant="caption">Showing {filtered.length} of {summary.total} projects</Typography>`.